



Project Report On

Structured Healthcare Data Retrieval via Natural Language and Predictive Analysis

*Submitted in partial fulfillment of the requirements for the
award of the degree of*

Bachelor of Technology

in

Computer Science and Engineering

By

Reno Jacob Zachariah (U2203176)

Reuben Mathew Shaji (U2203177)

Rohan Cijoy (U2203181)

Vyshnav A K (U2203214)

Under the guidance of

Dr. Amitha Mathew

**Department of Computer Science and Engineering
Rajagiri School of Engineering & Technology (Autonomous)
(Parent University: APJ Abdul Kalam Technological University)**

Rajagiri Valley, Kakkanad, Kochi, 682039

March 2026

CERTIFICATE

*This is to certify that the main project report entitled "**Structured Healthcare Data Retrieval via Natural Language and Predictive Analysis**" is a bonafide record of the work done by **Reno Jacob Zachariah (U2203176)**, **Reuben Mathew Shaji (U2203177)**, **Rohan Cijoy (U2203181)**, **Vyshnav A K (U2203214)** submitted to the APJ Abdul Kalam Technological University in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology (B. Tech.) in Computer Science and Engineering during the academic year 2025-2026.*

Project Guide

Dr. Amitha Mathew
Assistant Professor
Dept. of CSE
RSET

Project Coordinator

Ms. Jisha Mary Jose
Assistant Professor
Dept. of CSE
RSET

Head Of Department

Dr. Preetha K G
Professor
Dept. of CSE
RSET

ACKNOWLEDGEMENT

We wish to express our sincere gratitude towards **Rev. Dr. Jaison Paul Mulerikkal CMI**, Principal of RSET, and **Dr. Preetha K.G**, Head of the Department and Professor, Computer Science and Engineering for providing us with the opportunity to undertake our main project, "Structured Healthcare Data Retrieval via Natural Language and Predictive Analysis".

We are highly indebted to our project coordinator, **Ms. Jisha Mary Jose**, Asst.Professor, Department of Computer Science and Engineering for her valuable support.

It is indeed our pleasure and a moment of satisfaction for us to express our sincere gratitude to our project guide **Dr. Amitha Mathew**, Asst.Professor, Department of Computer Science and Engineering for her patience and all the priceless advice and wisdom she has shared with us. The authors gratefully acknowledge **SkillGenX Innovations Pvt. Ltd.** for providing the opportunity to work on this industry project, for their technical support and guidance and for granting permission to publish this research.

Last but not the least, We would like to express our sincere gratitude towards all other teachers and friends for their continuous support and constructive ideas.

Reno Jacob Zachariah

Reuben Mathew Shaji

Rohan Cijoy

Vyshnav A K

Abstract

The effective utilization of Electronic Health Records (EHR) in critical care is often hindered by two significant bottlenecks: the high technical barrier of structured query languages like SPARQL for clinical practitioners, and the "static" nature of traditional Knowledge Graphs (KGs) which fail to capture the temporal vital patterns of patient deterioration. This research presents an integrated solution that addresses both the NL to SPARQL translation challenge and the requirement for proactive predictive analytics over temporal Knowledge Graph. On the retrieval front, the system has an intelligent agent using LangChain and Large Language Models (LLMs) to map natural language queries into precise, executable SPARQL. This enables non-technical healthcare staff to perform complex, structured data retrieval from an RDF-based Knowledge Graph (Apache Jena Fuseki). To address the temporal limitation, the architecture incorporates a Long Short-Term Memory (LSTM) neural network that operates on a sliding-window basis over incoming vital sign streams. Unlike static retrieval, this approach captures the trajectory of physiological change, allowing for the early detection of time-sensitive conditions such as Sepsis, Hypoxia, and Hypertensive Crises. The system's uniqueness lies in its semantic feedback loop: generated predictions are re-inserted into the Knowledge Graph allowing them to be immediately retrievable via natural language.

Contents

Acknowledgements	i
Abstract	ii
List of Abbreviations	vi
List of Figures	vii
List of Tables	viii
1 Introduction	1
1.1 Background	1
1.2 Problem Definition	2
1.3 Scope and Motivation	2
1.4 Objectives	3
1.5 Challenges	3
1.6 Assumptions	4
1.7 Societal / Industrial Relevance	5
1.8 Summary of the chapter	6
2 Literature Survey	7
2.1 Review of Key Research Works	8
2.1.1 Querying Knowledge Graphs in Natural Language	8
2.1.2 Predicting Future Disorders via Temporal Knowledge Graphs and Medical Ontologies	9
2.1.3 Knowledge-Guided Time-Aware LSTM for Continuous Clinical Risk Prediction	10
2.1.4 Large Language Models for Knowledge Graph Question Answering	11
2.2 Comparative Analysis	11

2.3	Summary and Research gaps	12
3	System Design	14
3.1	Overall System Architecture	14
3.2	Module Diagrams and Descriptions	16
3.2.1	Synthesizer Module	16
3.2.2	SPARQL Executor Module	17
3.2.3	NL to SPARQL Agent Module	18
3.2.4	Real time Risk Prediction Module	20
3.2.5	API Gateway Module	21
3.2.6	Mini Apps (UI) Module	23
3.3	Design Diagrams	25
3.3.1	Use Case Diagram	25
3.3.2	Sequence Diagram	26
3.4	Project Timeline	27
3.5	Work Breakdown and Responsibilities	27
3.6	Key Deliverables	28
3.7	Summary of the chapter	29
4	Implementation Details	30
4.1	Overview	30
4.2	Knowledge Graph Construction and Data Processing	30
4.3	Natural Language to SPARQL Retrieval Pipeline	31
4.4	Real Time Predictive Analytics Framework	32
4.4.1	Inference Pipeline	33
4.4.2	Risk Prediction Outcomes	33
4.4.3	Alert Generation and Notification	34
4.5	Software and Hardware Requirements	34
4.5.1	Hardware Requirements	34
4.5.2	Software Requirements	34
4.5.3	Dataset	35
4.6	Summary of the chapter	35

5	Results and Discussions	36
5.1	Overview	36
5.2	UI Design	37
5.3	Evaluation Tables and Metrics	40
5.4	Discussion	42
5.5	Summary of the chapter	44
6	Conclusion	45
6.1	Conclusion	45
6.2	Future Scope	46
6.3	Summary of the chapter	46
	References	47
	Appendix A: Presentation	50
	Appendix B: Vision, Mission, Programme Outcomes and Course Outcomes	79
	Appendix C: CO-PO-PSO Mapping	83

List of Abbreviations

NLI	– Natural Language Interface
NLP	– Natural Language Processing
NL	– Natural Language
KG	– Knowledge Graph
RDF	– Resource Description Framework
SPARQL	– SPARQL Protocol and RDF Query Language
EHR	– Electronic Health Records
ETL	– Extract, Transform, Load
LLM	– Large Language Model
LSTM	– Long Short-Term Memory
NER	– Named Entity Recognition
JSON	– JavaScript Object Notation
API	– Application Programming Interface
TTL	– Turtle (RDF Serialization Format)
IDE	– Integrated Development Environment
CPU	– Central Processing Unit
RAM	– Random Access Memory
VS Code	– Visual Studio Code
NLI-KG	– Natural Language Interface for Knowledge Graph
NL-SPARQL	– Natural Language to SPARQL Conversion System
AI	– Artificial Intelligence
ML	– Machine Learning

List of Figures

3.1	Architecture Diagram	14
3.2	Synthesizer Module	16
3.3	SPARQL Executor Module	17
3.4	NL to SPARQL Agent Module	18
3.5	LSTM based Risk Prediction Module	20
3.6	API Gateway Module	21
3.7	Mini Apps Module	23
3.8	Use Case Diagram	25
3.9	Sequence Diagram	26
3.10	Project Timeline (Gantt Chart)	27
5.1	NL to SPARQL query result	37
5.2	Vital entry into the KG	38
5.3	Risk Alert	39
5.4	Confusion matrix of medical risk classification	41

List of Tables

2.1	Comparison of research studies related to healthcare knowledge graphs and predictive analytics	12
3.1	Work Breakdown and Responsibilities	28
5.1	Overall performance metrics of the classification model	40
5.2	Class-wise performance metrics	40
5.3	Notable misclassification summary	41
5.4	Example prediction cases	42

Chapter 1

Introduction

1.1 Background

Electronic Health Records (EHRs) keep a lot of information about patients, such as their diagnoses, lab results, medications, and important observations. These datasets have a lot of potential to make clinical decision-making better, allow for predictive healthcare analytics, and improve patient care outcomes. But even though there are big, well-structured datasets, it is still hard to get useful information from them.

To get information from most healthcare databases, you need to know how to use complicated query languages like SQL or SPARQL. Because of this, healthcare workers often need help from technical experts to get to patient records and make reports. Because of this reliance, it takes longer to get important information and it is harder to get useful clinical insights, especially when time is of the essence.

Recent progress in Natural Language Processing (NLP) and knowledge graph technologies makes it possible to close this gap. Knowledge Graphs (KGs) show data as linked entities and relationships. This makes them great for modeling complicated healthcare information like patients, diagnoses, treatments, and important observations. By combining natural language interfaces with knowledge graphs, doctors can get structured medical data just by asking questions in plain English.

Along with being able to get data, modern healthcare systems need predictive capabilities that can spot possible medical risks. Models that use machine learning, especially deep learning methods like Long Short-Term Memory (LSTM) networks, have done well at looking at medical data over time. These models can find patterns in a patient's vital signs over time and guess what clinical risks might happen, like sepsis, hypoxia, or high blood pressure.

The proposed system, called *Structured Healthcare Data Retrieval via Natural Lan-*

guage, wants to bring these technologies together into a single platform for clinical decision support. The system lets healthcare workers ask structured Electronic questions.

Natural language queries are used to get Health Record data. These queries are then automatically translated into SPARQL queries that run over a healthcare knowledge graph. The system also has a temporal prediction module that uses an LSTM-based model to look at patient vitals and make predictions about risk and alerts for possible health problems.

The proposed system combines natural language interfaces, knowledge graphs, and predictive analytics to give doctors quick and easy access to health information. It also helps with proactive patient monitoring.

1.2 Problem Definition

Healthcare professionals frequently need immediate access to patient data contained in Electronic Health Records (EHRs). But getting structured healthcare data usually requires knowledge of database query languages like SQL or SPARQL. This reliance on technical experts makes things harder for doctors, which slows down their access to important medical information. Moreover, current healthcare data retrieval systems mainly concentrate on querying historical data and do not possess integrated predictive analytics functionalities that can evaluate temporal patient information and detect potential health risks proactively.

So, there needs to be an intelligent system that lets healthcare professionals ask structured medical questions in plain language and also gives them predictive insights based on patient vitals and past clinical patterns.

1.3 Scope and Motivation

The project will make an integrated healthcare analytics system that uses natural language processing, knowledge graph technology, and machine learning-based predictive analytics.

The system lets doctors get patient information using natural language queries, so they don't have to learn complicated query languages. The information that was found comes from a healthcare knowledge graph made from structured clinical datasets.

The system also has a predictive analytics module that looks at a patient’s vital signs over time to find possible clinical risks. The system uses a sliding-window LSTM model to find patterns in vital signs over time. It then sends alerts when it finds patterns that are not normal and could mean health problems.

The impetus for this project stems from the increasing demand for intelligent clinical decision-support systems that enhance healthcare data accessibility and facilitate proactive patient monitoring. The system’s goal is to help healthcare professionals make decisions faster and with more information by combining data retrieval and predictive analytics on one platform.

1.4 Objectives

1. To create a natural language interface that lets doctors search healthcare databases using language that is common.
2. To automatically turn natural language questions into structured SPARQL questions that can be used to get information from healthcare knowledge graphs.
3. To make a healthcare knowledge graph that shows how patients, medical conditions, treatments, and important observations are all connected.
4. To set up a predictive analytics module that looks at patient vitals over time using a Long Short-Term Memory (LSTM) model.
5. To send out alerts in real time about possible health risks like sepsis, hypoxia, a crisis of high blood pressure, and a risk of falling.
6. To make it easy for healthcare professionals to use the system without needing to know how to use technology.

1.5 Challenges

The development of an intelligent healthcare data retrieval and prediction system presents several technical and practical challenges.

- **Natural Language Ambiguity:** Queries in natural language often have words that are unclear or don’t have enough context. Understanding what users want and

connecting it to the right entities and relationships in the knowledge graph is still a difficult job.

- **Schema Mapping Errors:** When the system translates natural language queries into SPARQL, it may not connect user terms to RDF predicates or classes in the knowledge graph schema correctly. These kinds of mapping mistakes can make queries that don't work or give no results.
- **Knowledge Graph Complexity:** Healthcare knowledge graphs are very complicated because they have a lot of different entities and relationships. These include patients, medical conditions, medications, and vital observations. To manage and query large graph structures well, you need to carefully design your schema and optimize how your queries run.
- **Temporal Data Processing:** To do predictive analysis with patient vitals, you need to work with time-series medical data. It is very hard to find useful patterns in temporal data while keeping the model stable.
- **Data Sparsity and Incomplete Records:** In real-world healthcare datasets, important measurements and clinical observations may not be there or may not be recorded consistently. This lack of data can make machine learning models work less well.
- **Model Sensitivity to Sampling Frequency:** Predictive models like LSTM networks are affected by the time between observations. Changes in sampling frequency might make it harder for the model to find long-term physiological patterns.

1.6 Assumptions

The proposed healthcare data retrieval and predictive analytics system's development and operation are predicated on the following assumptions:

1. **Availability of Structured Healthcare Data:** It is presumed that the Electronic Health Records (EHRs) utilized by the system are maintained in a structured format amenable to conversion into a healthcare knowledge graph featuring clearly delineated entities and relationships.

2. **Consistency of Knowledge Graph Schema:** The healthcare knowledge graph uses a consistent ontology schema to define medical entities like patients, diagnoses, medications, and vital observations using standard RDF predicates and classes.
3. **Valid Natural Language Queries:** The system expects users to ask meaningful and grammatically correct questions in natural language about health care information. This lets the NLP module correctly figure out what the user wants.
4. **Availability of Temporal Patient Data:** The predictive analytics module presumes that patient vital signs, including heart rate, blood pressure, oxygen saturation, and temperature, are systematically documented and preserved with precise timestamps.
5. **Consistent Sampling Frequency of Medical Data:** The LSTM-based prediction model presumes that the temporal patient data is documented at relatively uniform time intervals. The model's ability to learn over time may be affected by big changes in the sampling frequency.
6. **Reliable Data Preprocessing:** It is assumed that preprocessing steps like cleaning the data, normalizing it, and dealing with missing values are done correctly before the data is used to build a knowledge graph or do predictive analysis.
7. **Secure Access to Healthcare Data:** The system assumes that there are proper authentication and authorization systems in place to make sure that only healthcare professionals who have permission can see patient data.
8. **Clinical Decision Support Role:** The proposed system is meant to help healthcare professionals find information and spot possible risks by acting as a decision-support tool. Qualified healthcare professionals are still in charge of making final medical decisions.

1.7 Societal / Industrial Relevance

From a societal point of view, the system lets healthcare workers quickly get to patient information by using natural language queries. This lessens the need for technical staff and helps doctors get important information faster. Quicker access to medical data can

lead to more accurate diagnoses and quicker medical interventions, which will improve patient outcomes in the long run.

The system’s predictive analytics feature also sends out early warning alerts for possible medical risks like sepsis, hypoxia, a hypertension crisis, and the risk of falling. If doctors find these conditions early, they can start preventive treatment, which lowers the risk of complications and makes patients safer.

From a business point of view, hospitals and other healthcare organizations can add these smart systems to their clinical information infrastructure. Healthcare systems that use knowledge graphs organize medical data in a way that makes it easier to analyze and find. Adding machine learning models for predictive monitoring can also make hospital decision-support systems better and make operations run more smoothly.

The suggested system is a big step toward making smart healthcare information systems that help both doctors and healthcare organizations. It does this by putting together natural language interfaces, knowledge graphs, and predictive analytics.

1.8 Summary of the chapter

The growing number of Electronic Health Records has opened up a lot of possibilities for data-driven healthcare analytics. But to get to structured healthcare data, you often need to know how to use query languages, which makes it harder for doctors to get to.

This project suggests a system that lets clinicians get structured data by asking questions in plain English. The system uses a healthcare knowledge graph to show how patients, diagnoses, treatments, and important observations are all connected. When you type in a natural language query, it is automatically turned into a SPARQL query and run over the knowledge graph to find the information you need.

The system has a predictive analytics module that uses an LSTM-based model to look at patient vital data over time. This is in addition to data retrieval. This module sends out early warning alerts for possible health risks, giving doctors time to take action before things get worse.

The proposed system combines natural language processing, knowledge graph technology and predictive analytics based on machine learning to create an intelligent clinical platform that makes healthcare data easier to access and helps improve patient care.

Chapter 2

Literature Survey

As healthcare systems become more digital, more and more people are using Electronic Health Records (EHRs). This has led to the creation of huge amounts of structured and unstructured medical data. These records have information about the patient, such as their age, sex, as well as their diagnoses, medications, lab results, and vital signs that are monitored all the time. Even though this kind of data can help doctors make better decisions, it is still hard to find the right information in healthcare databases.

To get data from databases, traditional healthcare information systems usually need structured query languages like SQL or SPARQL [10]. These query languages require technical knowledge, which can be hard for healthcare workers who don't have a background in database management. Because of this, doctors often have to ask technical staff to get certain information, which can slow down access to important clinical insights.

Recent progress in Natural Language Processing (NLP), knowledge graph technologies, and machine learning has led to promising ways to make healthcare data easier to get to [1, 7]. Knowledge graphs show how healthcare entities like patients, diseases, medications, and medical procedures are connected through nodes and relationships. This makes it possible to do complex semantic queries. Knowledge graphs, when used with natural language interfaces, let doctors get structured information just by asking questions in everyday language [1].

In addition to getting data, predictive analytics has become a powerful tool in healthcare systems. Machine learning models can look at past patient data and find patterns that could mean health risks [9]. Long Short-Term Memory (LSTM) networks and other temporal deep learning models are very good at looking at sequential healthcare data [6]. They can even use patient monitoring records to predict clinical events [11].

This literature survey examines significant research contributions pertaining to natural language querying of knowledge graphs, the construction of healthcare knowledge graphs,

temporal knowledge graphs for disease prediction, and machine learning models for clinical risk prediction [8, 4]. These studies lay the groundwork for the creation of intelligent healthcare systems that integrate natural language interfaces with predictive analytics to facilitate clinical decision-making.

2.1 Review of Key Research Works

2.1.1 Querying Knowledge Graphs in Natural Language

The research titled “Querying knowledge graphs in natural language” introduces an innovative Question-Answering (QA) system aimed at converting natural language inquiries into SPARQL queries [1]. The research examines the primary issue that knowledge graphs are generally vast and necessitate specialized technical expertise in query languages and RDF data models, thereby excluding non technical end users. The system is tested with two well known benchmarks: the 7th Question Answering over Linked Data Challenge (QALD-7) and the Large-Scale Complex Question Answering Dataset (LC-QuAD) [1]. There are 5,000 question SPARQL pairs in LC-QuAD that cover more than 5,000 resources and 600 properties. Some of the questions are very hard and involve counts and booleans. QALD-7 is a set of real world tests for QA systems that work with linked data. The Resource Description Framework (RDF) organizes the data so that it is made up of subject–predicate–object triples [10]. This modular approach makes it possible to use the system on open domain knowledge graphs like DBpedia by linking natural language phrases to specific graph entities [1].

The proposed system divides the translation process into five smaller, more manageable tasks that are all part of a modular pipeline [1]. There are two types of parts in the architecture: those that don’t depend on the knowledge graph and those that do. When someone asks a question, the Question Analysis module does syntactic parsing, lemmatization, and dependency parsing. Then, a Question Type Classification module uses a Random Forest model to figure out if the question is a ”List,” ”Count,” or ”Boolean” type. Phrase Mapping (the step that depends on the graph) uses a group of tools, such as DBpedia Spotlight and Falcon, to map entities and relationships to the graph [1]. Then, Query Generation uses these mappings to make possible SPARQL triples. Finally, a Query Ranking module uses a Tree structured Long Short Term Memory (Tree-LSTM)

network to put candidate queries in order based on how similar the question’s dependency tree is to the query’s tree representation [1].

The system performs much better than the best models, as shown by the experimental results [1]. It shows a 15% improvement on the QALD-7 dataset and a 48% improvement on the LC-QuAD dataset. The system got an F-score of 0.68 and a precision of 0.88 on the LC-QuAD dataset. Despite its performance, the system faces a bottleneck in the phrase mapping step, where required properties cannot always be detected if they are implicitly stated in the question [1]. Additionally, the recall for ‘Boolean’ and ‘Count’ questions remains lower than ‘List’ questions due to the high complexity of the required SPARQL patterns and the limited size of available training data for these specific types.

2.1.2 Predicting Future Disorders via Temporal Knowledge Graphs and Medical Ontologies

The research paper ”Predicting Future Disorders via Temporal Knowledge Graphs and Medical Ontologies. is another important addition to healthcare analytics [8]. This study concentrates on forecasting future medical conditions through the examination of temporal relationships among clinical events in patient histories. The research employs patient medical histories that encompass diagnoses, treatments, medications, and laboratory observations documented over time [8]. In a temporal knowledge graph, each clinical event is shown as a relationship with a time stamp. Temporal knowledge graphs are different from regular knowledge graphs because they include time as an extra dimension [4]. This lets the system predict how the condition of a patient will change over time.

For example:

Patient → diagnosedWith → Diabetes (2019)

Patient → prescribedMedication → Insulin (2020)

Patient → developedCondition → KidneyDisease (2022)

These kinds of time-based relationships let the system look at how diseases get worse over time. The suggested method uses medical ontologies and temporal knowledge graph embeddings together [8]. Medical ontologies give diseases, symptoms, and treatments structured definitions that help the system understand how medical concepts are related to each other. We use graph embedding techniques to turn the knowledge graph into numerical vector representations [8]. These embeddings show both the structural links

between entities and the patterns over time in patient histories. Then, these embeddings are used to train machine learning algorithms to guess how likely it is that someone will get sick in the future [8].

Experimental evaluations demonstrate that temporal knowledge graph models surpass conventional static prediction models [8]. The system effectively discerns patterns in disease progression and forecasts potential medical conditions earlier than traditional methods. The main problem with modeling temporal knowledge graphs is that it's hard to keep track of big graphs with relationships that change over time [8]. The more temporal events there are, the more complicated the math gets.

2.1.3 Knowledge-Guided Time-Aware LSTM for Continuous Clinical Risk Prediction

The paper "Knowledge Guided Time Aware LSTM for Continuous Clinical Risk Prediction (KIT-LSTM)" talks about a deep learning model that can look at patient data over time and make predictions about clinical risks all the time [6]. The model learns from long term patient records that come from electronic health records [6]. Heart rate, blood pressure, oxygen saturation, temperature, and respiratory rate are all important signs that are kept in these records.

The suggested model builds on the standard Long Short Term Memory (LSTM) architecture by adding medical ontologies' domain knowledge [6]. In typical LSTM models, input variables are handled separately. However, the KIT LSTM architecture incorporates relationships between medical variables. The model also has a time aware gating mechanism that changes how much past observations affect the current ones based on how much time has passed between events [6]. This is especially helpful for working with healthcare data that isn't sampled regularly.

Experimental results show that the KIT LSTM model makes predictions much more accurately than traditional machine learning models like logistic regression and random forests [6]. The incorporation of domain knowledge enables the model to identify clinically significant relationships among medical variables. Even though the model makes very accurate predictions, deep learning models often need a lot of data to train on and can be expensive to run [6].

2.1.4 Large Language Models for Knowledge Graph Question Answering

Recent progress in AI has led to the use of big language models in knowledge graph question answering systems [3, 2, 5]. With these systems, people can use conversational queries to get structured information from knowledge graphs. These approaches aim to bridge the gap between unstructured human language and structured semantic data, enabling non technical users to interact with complex knowledge bases without requiring expertise in query languages such as SPARQL.

Using transformer-based architectures, large language models look at natural language questions [3]. These models find the right entities and relationships in the knowledge graph schema. Entity linking techniques are used to connect words in the query to specific entities in the knowledge graph [2]. After mapping the entities, the system makes a SPARQL query that gets the information it needs. In addition, recent frameworks integrate retrieval augmented generation and reasoning mechanisms to improve the accuracy of query generation and reduce ambiguity in mapping user queries to graph structures [5].

LLM-based question-answering systems make knowledge graphs much easier to use by letting people talk to structured data [5]. They improve accessibility, enhance user interaction, and enable faster extraction of insights from large-scale healthcare and biomedical datasets. But LLM based systems can sometimes make up relationships that aren't in the knowledge graph schema [3]. This is still a big problem for real world use. Furthermore, ensuring factual correctness, reducing hallucinations, and maintaining alignment with the underlying knowledge graph remain key challenges that must be addressed before these systems can be reliably deployed in critical domains such as healthcare.

2.2 Comparative Analysis

The studies that were looked at show how important it is to combine natural language processing, knowledge graph technologies, and machine learning models to improve healthcare data analytics [1, 8, 6].

Table 2.1: Comparison of research studies related to healthcare knowledge graphs and predictive analytics

Paper	Focus Area	Data Type	Methodology	Advantages	Limitations
Structured Healthcare Data Retrieval via Natural Language [1, 7]	Healthcare data access	Electronic Health Records	NLP + Knowledge Graph + SPARQL	Enables natural language querying of medical data	Ambiguity in query interpretation
Predicting Future Disorders via Temporal Knowledge Graphs [8]	Disease prediction	Patient medical histories	Temporal Knowledge Graph Embeddings	Captures temporal relationships in medical data	High computational complexity
KIT-LSTM for Clinical Risk Prediction [6]	Clinical risk prediction	Temporal patient monitoring data	Knowledge-guided LSTM	Handles irregular time-series medical data	Requires large training datasets
Temporal Knowledge Graph Extraction [4]	Health risk prediction	Clinical event records	Graph construction + embeddings	Models disease progression over time	Difficult to scale for large datasets
Natural Language to SPARQL Systems [10]	Semantic data retrieval	Knowledge graph datasets	NLP + Query generation	Improves accessibility of semantic databases	Schema mapping errors
Knowledge Graph Question Answering Systems [7]	Intelligent data retrieval	Healthcare knowledge graphs	NLP + Graph reasoning	Provides interpretable answers from graph data	Requires well-structured knowledge graphs
LLM-based Knowledge Graph QA [3, 2, 5]	Conversational data retrieval	Large knowledge graphs	Transformer-based models	Enables conversational interaction with data	Hallucination and incorrect mappings

2.3 Summary and Research gaps

The literature review shows that knowledge graph technologies, natural language interfaces, and machine learning models are being used more and more to improve health care

data analytics [1, 7, 9]. Knowledge graphs offer a structured depiction of medical entities and their interrelations, whereas natural language interfaces allow clinicians to access information without necessitating technical query proficiency [1].

However, several challenges remain:

- It's hard to understand what someone means when they use natural language because it often has more than one meaning [1].
- Query generation systems based on LLMs might make wrong schema mappings [3].
- A lot of predictive healthcare models don't use structured medical knowledge from knowledge graphs [8, 4].
- Most current systems only focus on either data retrieval or predictive analytics, not both [5].

Chapter 3

System Design

This chapter presents how the proposed clinical knowledge graph based healthcare monitoring system is put together and designed. It tells you how different parts work including the NL-to-SPARQL query agent, the Knowledge Graph, real-time risk prediction module and user interface applications work together to make a complete and functional framework. The chapter also talks about the system modules, the architectural workflow, the software components and how they work together, giving a comprehensive understanding of how the system is designed and deployed

3.1 Overall System Architecture

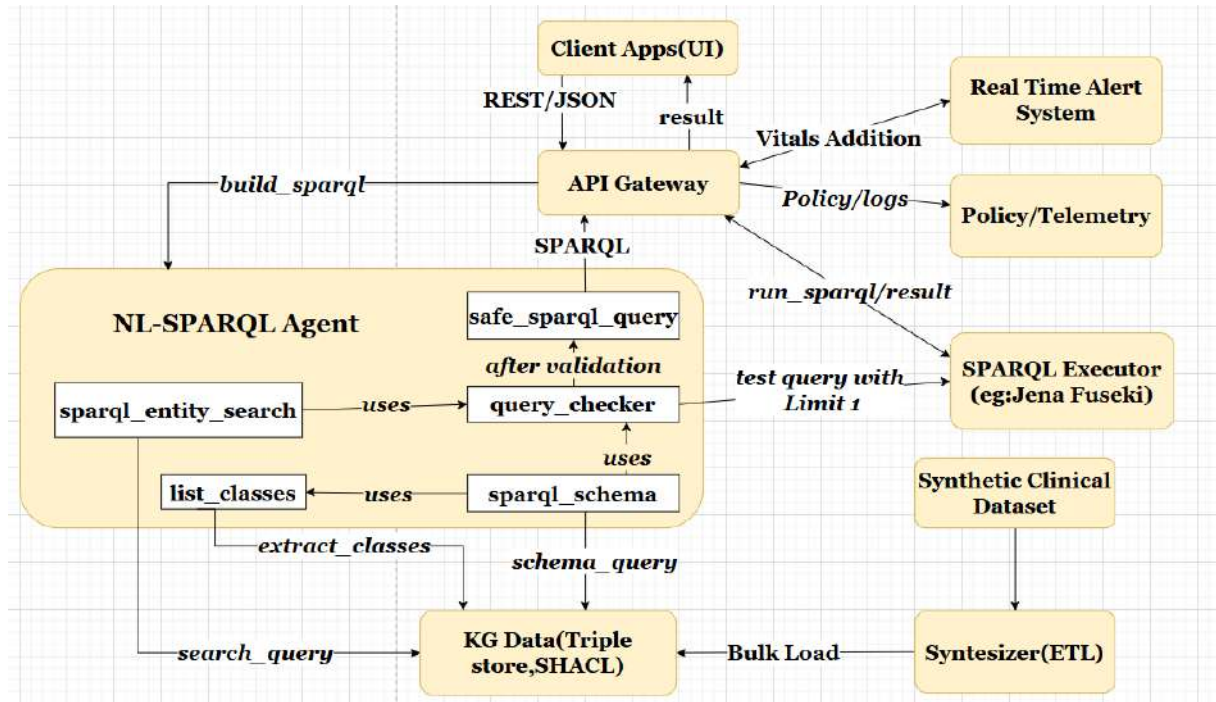


Figure 3.1: Architecture Diagram

The proposed system architecture, as shown in Figure 4.1, integrates natural language query processing, knowledge graph-based data retrieval, and real-time clinical risk monitoring into a unified healthcare analytics framework. It enables clinicians to interact with structured healthcare data using intuitive natural language queries while simultaneously supporting continuous monitoring of patient vital parameters for proactive decision-making.

User requests are initiated through client applications and routed via an API Gateway, which manages communication between the frontend and backend services using RESTful APIs and JSON formats. The gateway ensures secure and efficient data exchange while handling request validation and routing. Natural language queries are processed by the NL-to-SPARQL agent, which interprets user intent, extracts relevant entities, and generates corresponding SPARQL queries using contextual knowledge such as ontology schema and predefined templates.

The generated queries are validated to ensure they adhere to safety constraints such as read-only operations and query limits before being executed by the SPARQL execution engine. These queries interact with a triple store (e.g., Apache Jena Fuseki) that hosts the healthcare knowledge graph. The knowledge graph is constructed from a synthetic clinical dataset using an ETL pipeline that converts structured tabular data into RDF triples, enabling efficient semantic querying and relationship-based data retrieval.

In addition to data retrieval, the system supports real-time analysis of patient vitals. New data entries trigger a predictive module that analyzes temporal patterns to detect potential health risks such as sepsis, hypoxia, and hypertension. Alerts are generated based on prediction outcomes to assist clinicians in timely intervention. Logging and telemetry components are also integrated to monitor system performance, maintain audit trails, and ensure reliability and scalability.

Furthermore, the modular design of the architecture ensures scalability and flexibility, allowing for the integration of additional analytical or predictive components with minimal system disruption.

3.2 Module Diagrams and Descriptions

3.2.1 Synthesizer Module

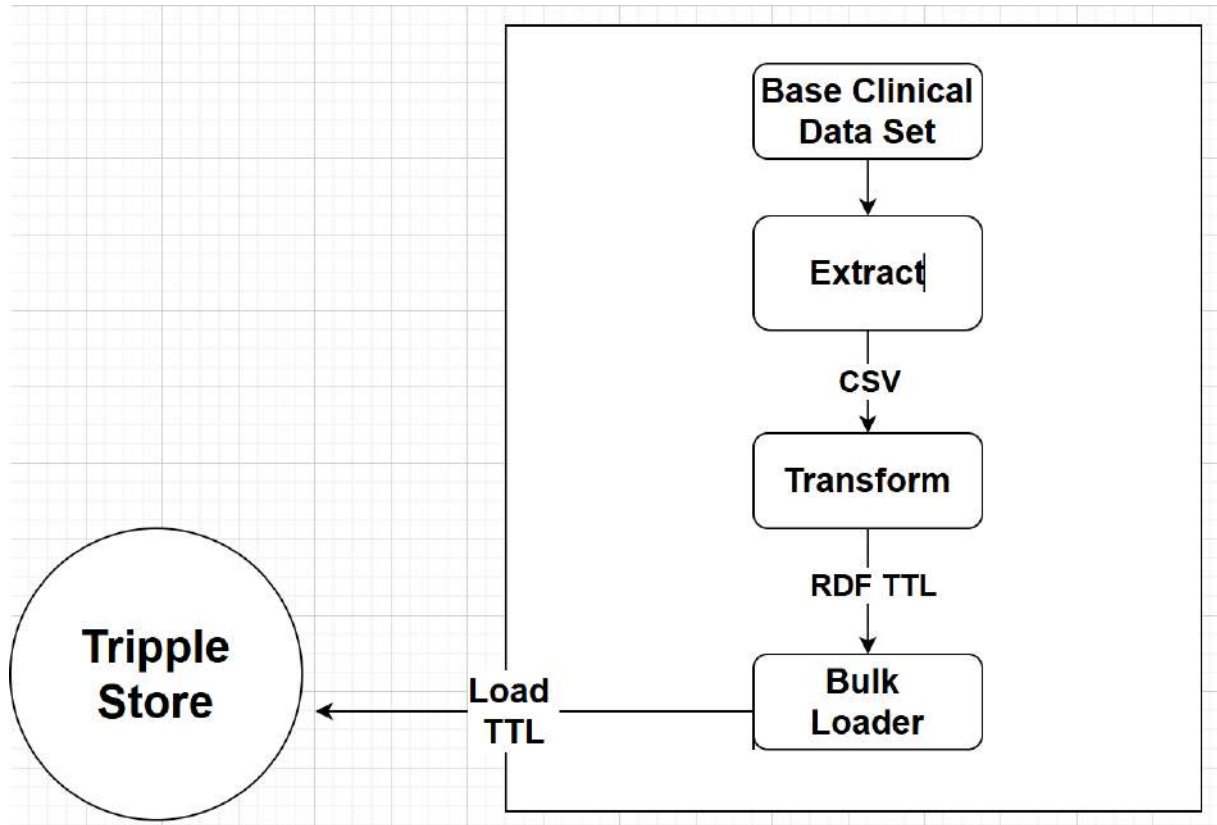


Figure 3.2: Synthesizer Module

Figure 3.2 depicts the synthesizer module responsible for generating structured clinical data and converting it into a knowledge graph format that can be queried using SPARQL.

- Uses the base clinical dataset as the foundation for creating an extended synthetic representation of hospital data.
- Generate multiple CSV files representing different aspects of the hospital information, including patient records, hospital stays and vital sign observations.
- Implements an Extract-Transform-Load (ETL) pipeline that processes the generated CSV files and converts them into RDF triples.
- During the transformation stage, the CSV data is mapped into RDF triples that represent entities such as patients, hospital admissions, vital observations, doctors

and hospital resources.

- The transformed RDF data is stored in `.ttl` (Turtle) format files, which provide a structured semantic representation of the clinical dataset.
- These Turtle files are then bulk loaded into the triple store for efficient semantic querying and retrieval through SPARQL.

The synthesizer module enables the system to construct a scalable knowledge graph from structured clinical data, allowing the NL-SPARQL agent to retrieve relevant health-care information efficiently.

3.2.2 SPARQL Executor Module

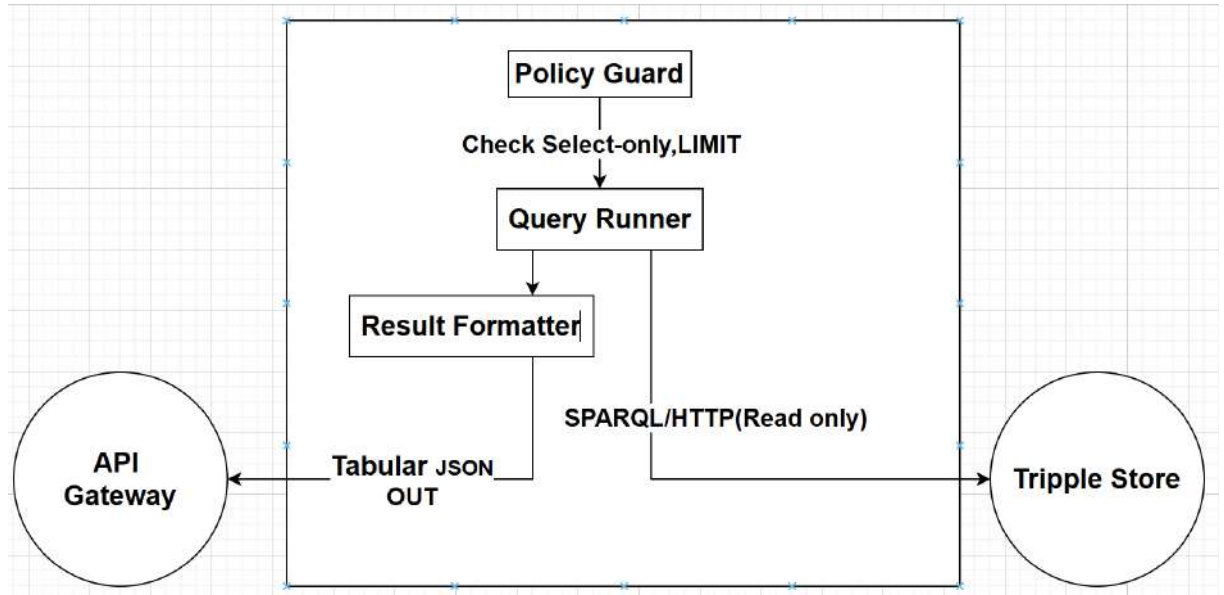


Figure 3.3: SPARQL Executor Module

Figure 3.3 depicts the SPARQL Executor module that is responsible for safely executing SPARQL queries generated from natural language questions on the knowledge graph.

- Receives SPARQL queries generated by the NL-SPARQL agent through the API gateway for execution on the knowledge graph.
- Implements a policy guard that validates queries to ensure they follow safe execution rules.

- Checks that all queries are **SELECT**-only operations and prevents any update or modification queries to protect harmful manipulation of the triple store.
- Automatically adds **LIMIT** constraints to queries in order to restrict excessive data retrieval and prevent system overload.
- Executes validated SPARQL queries against the triple store using SPARQL endpoints such as GraphDB or Apache Jena Fuseki.
- Formats the query results into a structured tabular JSON representation that can be easily processed by the NL-SPARQL agent and displayed by client applications.

The SPARQL Executor module ensures secure and efficient access to the knowledge graph by enforcing query policies, preventing harmful operations, and providing structured results for downstream components of the system.

3.2.3 NL to SPARQL Agent Module

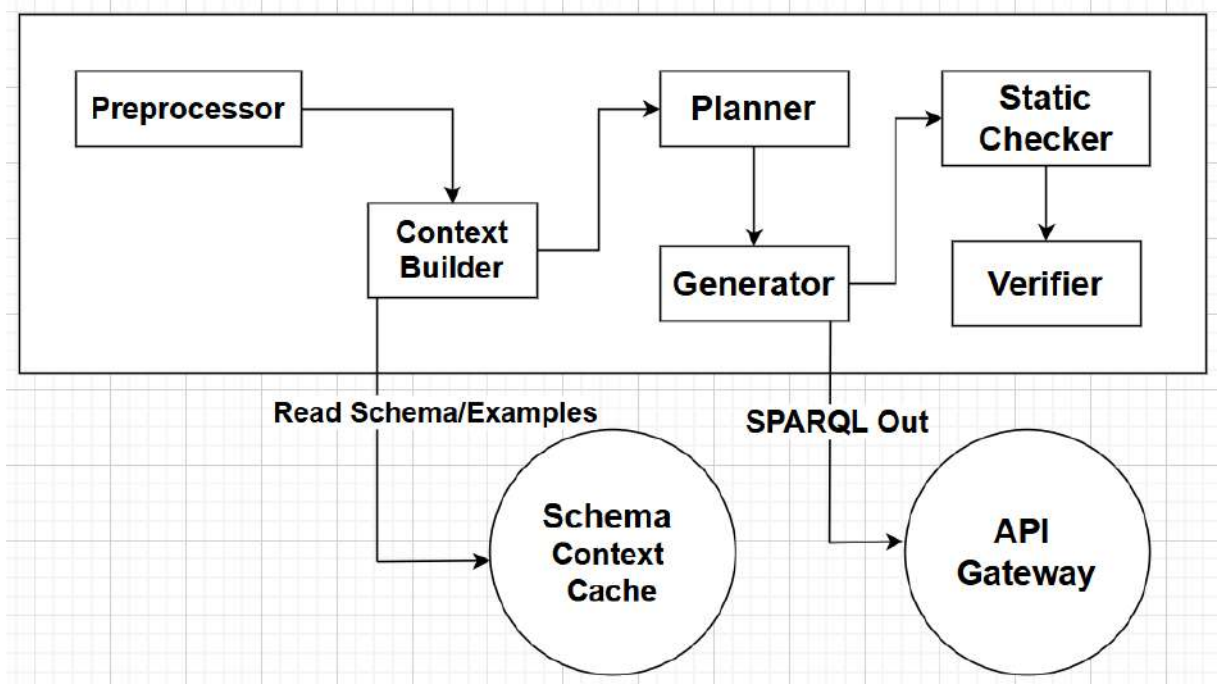


Figure 3.4: NL to SPARQL Agent Module

The NL to SPARQL agent module shown in Figure 3.4 is responsible for translating natural language questions into executable SPARQL queries that can be run on the healthcare knowledge graph.

- The agent is composed of five main tools: `safe_sparql_query`, `query_checker`, `sparql_schema`, `sparql_entity_search`, and `list_classes`, which together assist in constructing safe and accurate SPARQL queries.
- The preprocessing stage consists of the `list_classes` and `sparql_schema` tools. The `list_classes` tool retrieves the available ontology classes from the knowledge graph, while the `sparql_schema` tool provides schema-level information about entities and relationships within the graph.
- A context builder component utilizes this schema information along with system instructions and few-shot natural language to SPARQL examples to construct the prompt context required for query planning.
- Based on the generated context, the planner and generator components produce the corresponding SPARQL query that represents the user’s natural language request.
- The generated query is then passed through a static checker and verifier stage that uses the `query_checker` and `safe_sparql_query` tools to validate the query.
- These validation steps ensure that the generated query is read-only, contains proper prefixes, and includes appropriate `LIMIT` clauses to prevent excessive data retrieval.
- Schema-aware query construction ensures that only valid classes and relationships defined in the ontology are used during query generation.
- Error handling mechanisms are incorporated to detect incomplete or ambiguous queries and trigger refinement for improved correctness.
- The overall pipeline ensures efficient query execution by generating optimized queries that reduce unnecessary data retrieval and improve response time.

The NL-SPARQL agent enables users to interact with the healthcare knowledge graph using natural language while ensuring that the generated queries are syntactically correct, semantically meaningful, and safe for execution.

3.2.4 Real time Risk Prediction Module

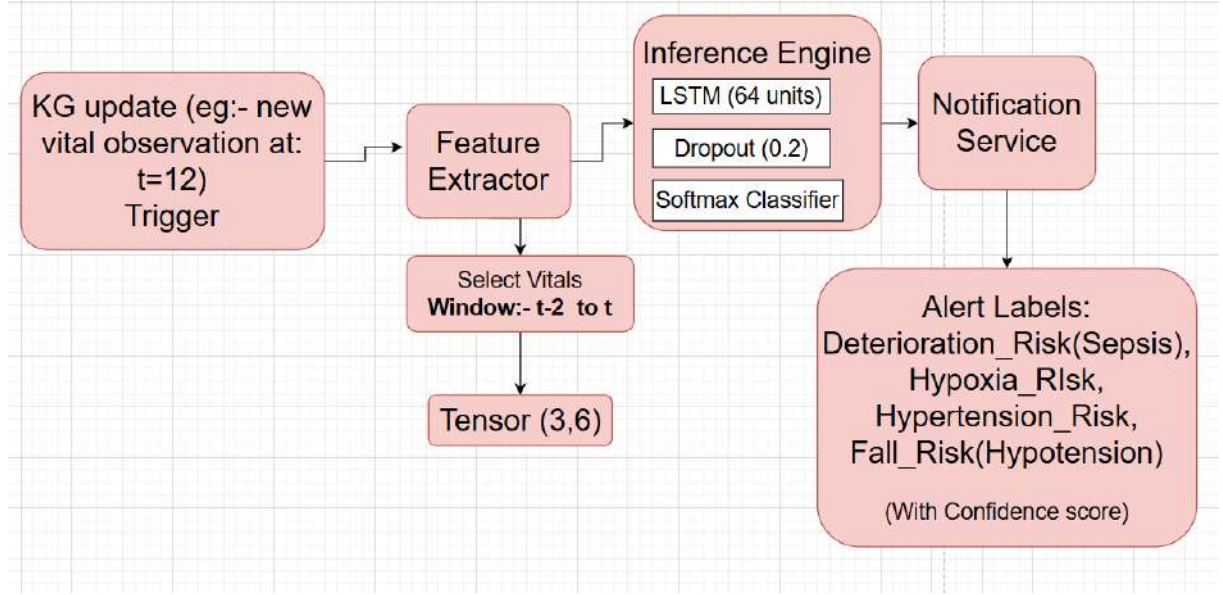


Figure 3.5: LSTM based Risk Prediction Module

Figure 3.5 shows LSTM based real-time clinical risk prediction using temporal patient vital data and generates alerts for potential medical deterioration.

- The module is triggered in an event-driven manner when updates occur in the Knowledge Graph (KG), such as the insertion of a new vital observation (e.g., a measurement recorded at time step $t = 12$).
- Once triggered, the feature extraction component retrieves patient vital observations from a temporal window ranging from $t - 2$ to t in order to capture recent physiological trends.
- These selected vitals are transformed into a structured tensor representation of size $(3, 6)$, where the rows represent temporal observations and the columns correspond to different vital parameters.
- The tensor is then passed to the inference engine, which uses a Long Short Term Memory (LSTM) neural network with 64 hidden units to learn temporal dependencies and detect patterns associated with clinical deterioration.

- A dropout layer with a rate of 0.2 is applied during inference to improve generalization and reduce the risk of overfitting.
- The final prediction stage uses a Softmax classifier to generate probabilistic outputs representing the likelihood of different clinical risk conditions.
- Based on these probabilities, the system generates alert labels such as *Deterioration Risk (Sepsis)*, *Hypoxia Risk*, *Hypertension Risk*, and *Fall Risk (Hypotension)*.
- The notification service dispatches alerts along with associated confidence scores, enabling clinicians to identify and respond to potential patient risks in a timely manner.

The real time risk prediction module facilitates continuous monitoring of patient vitals and provides early warnings of critical health conditions by using temporal deep learning models.

3.2.5 API Gateway Module

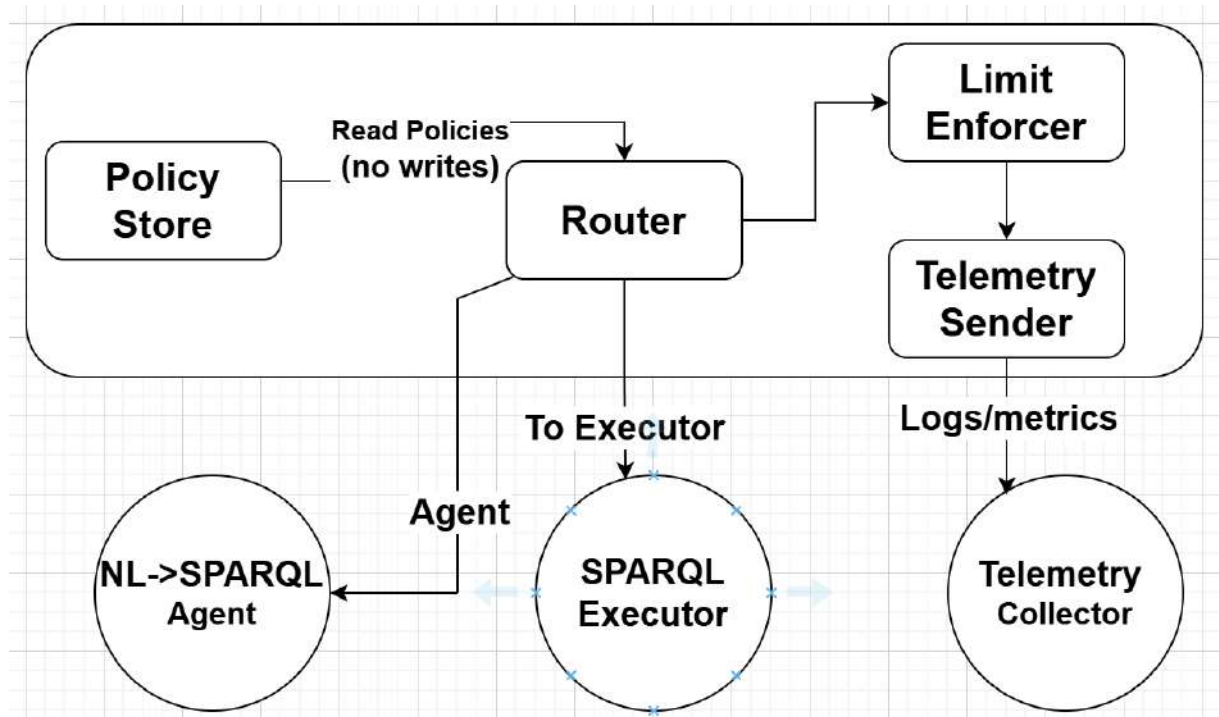


Figure 3.6: API Gateway Module

Figure 3.6 depicts the API Gateway module which is responsible for system security, request routing and operational monitoring to ensure safe and reliable execution of queries within the clinical knowledge graph.

- The module enforces strict security policies using a dedicated *Policy Store*, which maintains predefined rules that ensure only read-only access to the knowledge graph and prevents any unauthorized data modification.
- A central *Router* component manages incoming requests and directs them to the appropriate processing components, enabling efficient coordination between specialized agents and the SPARQL execution engine.
- The *NL to SPARQL Agent* processes natural language queries submitted by users and converts them into structured SPARQL queries that can be executed against the knowledge graph.
- The *SPARQL Executor* receives validated queries from the router and performs the final execution against the underlying triple store to retrieve the requested semantic data.
- To maintain system stability, a *Limit Enforcer* regulates query traffic and applies constraints such as query limits and resource thresholds to prevent system congestion and excessive load.
- For operational transparency, a *Telemetry Sender* continuously generates logs and performance metrics related to query processing and system activity.
- These logs and metrics are transmitted to a centralized *Telemetry Collector*, enabling monitoring, auditing and performance analysis of the entire system.

The security and policy management module ensures that all system operations are executed safely, efficiently, and transparently while maintaining strict access control and monitoring capabilities.

3.2.6 Mini Apps (UI) Module

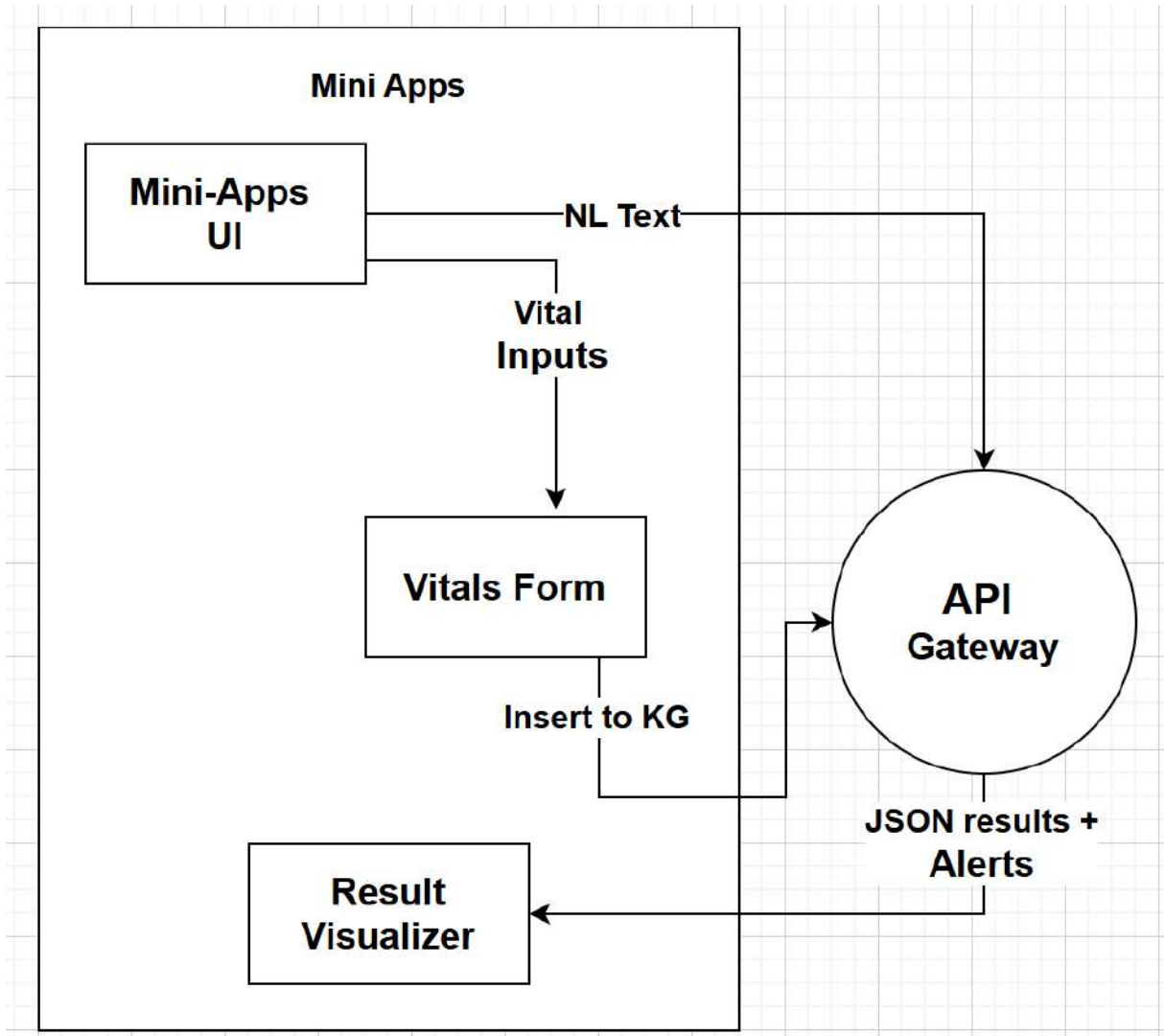


Figure 3.7: Mini Apps Module

This module provides the user interface that enables interaction with the system for both querying clinical knowledge and managing and adding patient vital information.

- The user interface is composed of two primary components: (1) *NL to SPARQL Agent Chat Interface* and (2) *Vitals Addition and Alert System*, ensuring both data retrieval and data entry functionalities are seamlessly integrated.
- The NL to SPARQL chat interface allows users to submit natural language queries related to clinical data. These queries are processed by the NL to SPARQL agent

and the final answers retrieved from the knowledge graph are displayed to the user in an intuitive format.

- The chat interface also supports iterative query refinement, allowing users to modify or rephrase their queries based on previous results to obtain more accurate and relevant information.
- The vitals management interface allows healthcare staff to admit a patient by creating a new hospital stay entry in the system.
- After admission, users can add patient vital observations such as heart rate, blood pressure, oxygen saturation, and other physiological measurements through the vitals form.
- The system ensures input validation and consistency by restricting data formats, thereby improving the reliability of stored clinical data.
- Each vital record is automatically timestamped with a predefined interval of four hours, ensuring consistent temporal data collection for downstream analytics and prediction tasks.
- The entered vitals are transmitted through the API Gateway and inserted into the Knowledge Graph (KG) for querying and analysis in real time.
- When sufficient temporal data becomes available, the LSTM based risk prediction module is triggered to analyze the latest observations.
- The interface dynamically updates to display newly generated predictions and alerts, ensuring that clinicians always have access to the most recent patient status without manual refresh.
- If a potential clinical risk is detected, the system generates notifications displaying the predicted risk state along with the confidence score.

The Mini Apps (UI) module enables smooth user interaction with the system by supporting natural language clinical queries, structured vital data entry, real-time alert visualization, and interactive data exploration.

3.3 Design Diagrams

3.3.1 Use Case Diagram

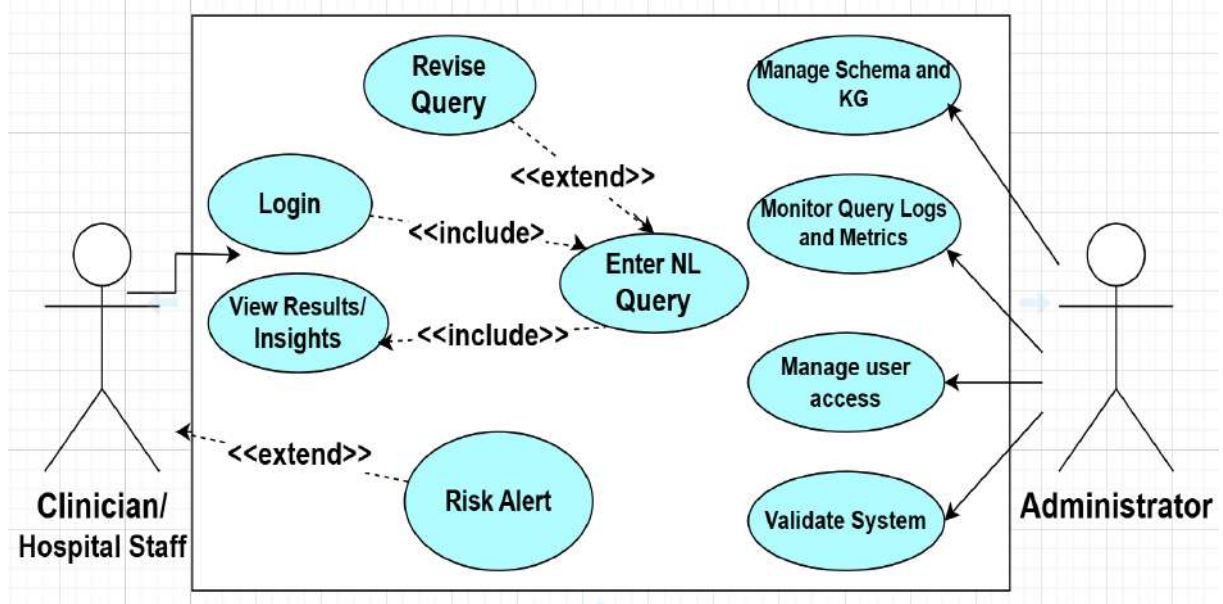


Figure 3.8: Use Case Diagram

Figure 3.2 gives the use case diagram of the system which involves two primary actors: **Clinician/Hospital Staff** and **Administrator**.

Clinician/Hospital Staff: Healthcare professionals use the system to retrieve patient insights through natural language queries. The clinician first logs into the system to ensure secure access. After authentication, the user can enter a natural language query to obtain structured healthcare information from the knowledge graph. If the returned information is not satisfactory, the user may revise the query for better results. The system then displays the retrieved results and insights. Additionally, when high risk medical patterns are detected, the system generates a *Risk Alert* to notify the clinician for timely medical intervention.

Administrator: The administrator is responsible for system maintenance and management. This includes managing the database schema and knowledge graph, monitoring system query logs and performance metrics, managing user access permissions and validating the system to ensure reliable operation.

Relationships: The diagram also illustrates relationships between use cases. The

Login process is included before users can access query-related functionalities. The *Revise Query* function extends the natural language query process, allowing users to refine their queries. Similarly, the *Risk Alert* extends the result generation process and is triggered only when high risk conditions are identified.

3.3.2 Sequence Diagram

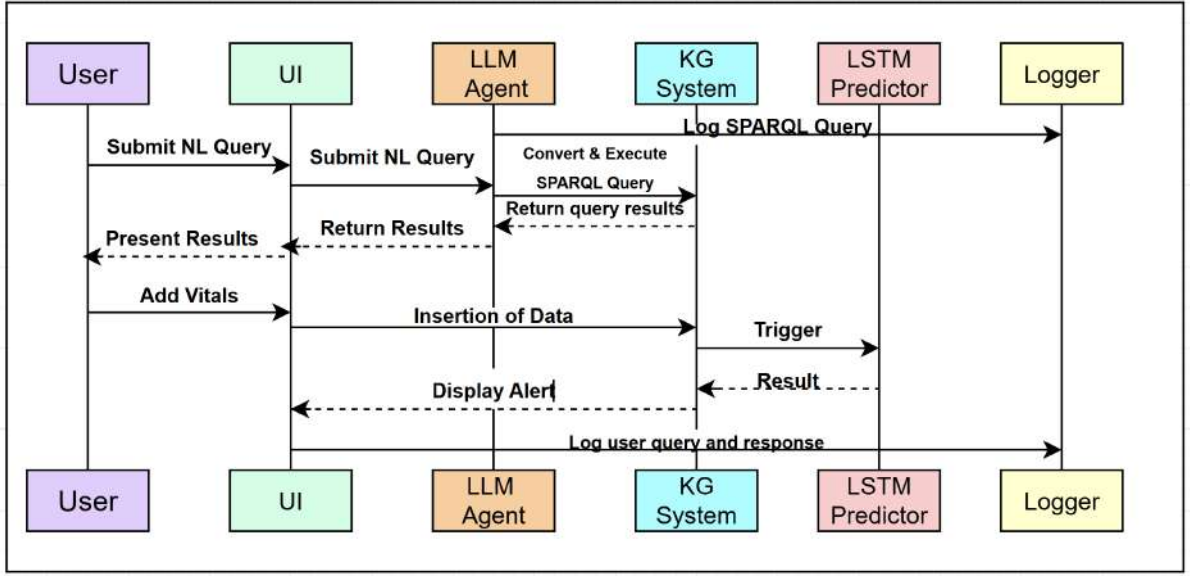


Figure 3.9: Sequence Diagram

The sequence diagram illustrates the interaction between the major components of the proposed healthcare data retrieval and prediction system. The process begins when the user submits a natural language (NL) query through the user interface (UI). The UI forwards the query to the LLM Agent, which interprets the query and converts it into a SPARQL query suitable for the knowledge graph. This query is then executed on the KG System, which retrieves the relevant healthcare data and returns the results to the LLM Agent. The processed results are then sent back to the UI and presented to the user.

In addition to query processing, the system also supports the insertion of new patient data. When the user adds vital parameters through the UI, the data is inserted into the KG System. This action triggers the LSTM Predictor, which analyzes the temporal patterns of the patient's vital signs to detect potential health risks. The prediction result is returned to the KG System, and if a potential risk is identified, an alert is displayed to the user through the UI.

Furthermore, the system maintains transparency and traceability by logging important system activities. The executed SPARQL queries, user queries and system responses are recorded by the Logger component. This logging mechanism helps in monitoring system performance, debugging issues and maintaining an audit trail of interactions within the system.

3.4 Project Timeline

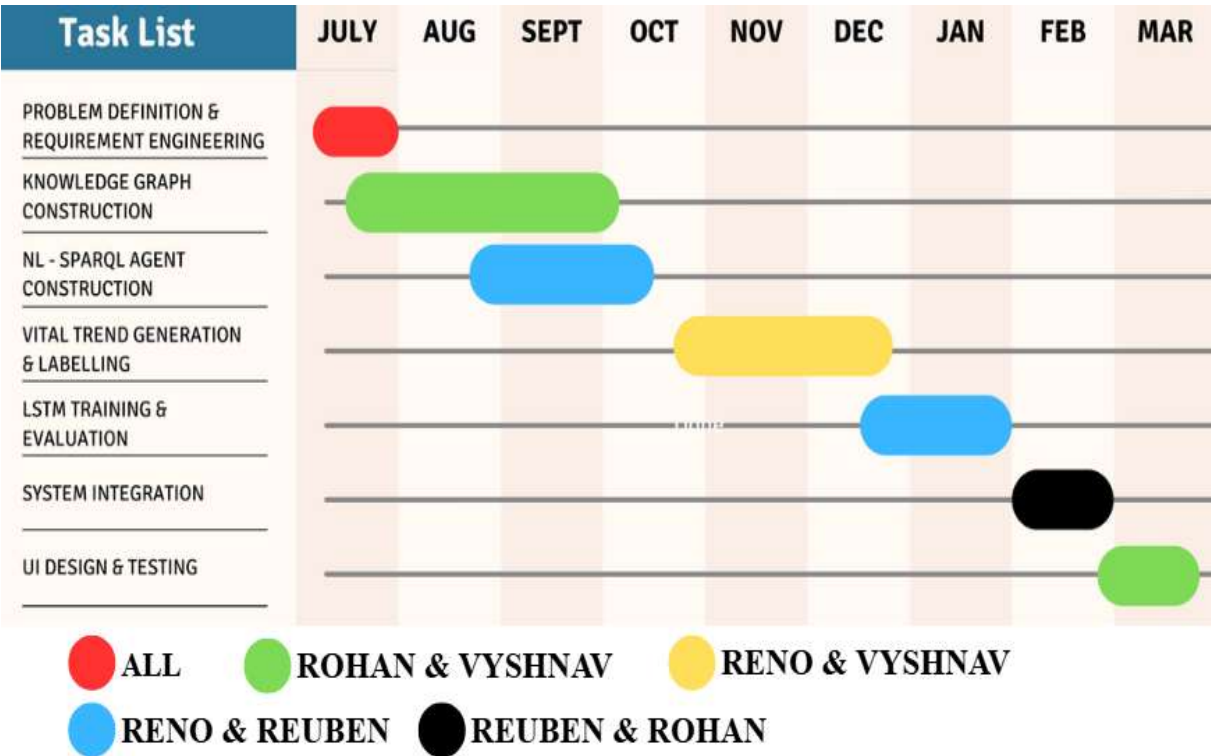


Figure 3.10: Project Timeline (Gantt Chart)

3.5 Work Breakdown and Responsibilities

The detailed work division among the team members is presented in Table 3.1.

Table 3.1: Work Breakdown and Responsibilities

Member	Module	Key Responsibilities
Reno Jacob Zachariah	NL to SPARQL & Model Training	Constructed NL to SPARQL agent, labeled vital observations with risk states, performed feature extraction and preprocessing for training, trained LSTM model, and conducted training evaluation.
Reuben Mathew Shaji	Backend, ETL & Integration	Prepared synthetic hospital and doctor datasets, developed ETL pipeline, integrated chatbot with UI, implemented logging system, and handled backend and API development.
Rohan Cijoy	UI & Agent Optimization	Prepared synthetic patient and stays data, developed user interface, tested NL to SPARQL agent, refined agent prompts based on testing, and implemented SHACL for validation.
Vyshnav A K	Knowledge Graph & Temporal Analysis	Prepared synthetic vital observations dataset, designed knowledge graph schema, generated vital trends for temporal analysis, tested data on Apache Jena Fuseki, and integrated LSTM for real-time alerts.

3.6 Key Deliverables

- A healthcare knowledge graph constructed from clinical data using an ETL pipeline, enabling structured semantic representation and efficient querying.
- An NL-to-SPARQL system that converts natural language queries into executable SPARQL queries for intuitive access to healthcare data.

- A secure SPARQL execution module that validates and executes queries on the knowledge graph while enforcing safety constraints.
- A real-time LSTM-based predictive model for analyzing patient vitals and identifying potential clinical risks.
- An alert generation system that notifies clinicians of predicted risks along with corresponding confidence scores.
- A user interface that supports natural language querying, patient data entry, and visualization of results and alerts.

3.7 Summary of the chapter

This chapter presented the system design and architecture of the proposed knowledge graph based healthcare monitoring system. It described how key components, including the NL-to-SPARQL agent, SPARQL executor, knowledge graph and real-time risk prediction module, are integrated to enable efficient clinical data retrieval and analysis.

The synthesizer module constructs the knowledge graph from structured clinical data, while the NL-SPARQL agent enables natural language interaction. The SPARQL executor ensures secure query execution, and the LSTM based prediction module supports real-time risk detection using patient vitals. The Mini Apps module provides user interaction, and the security module enforces safe and controlled system operations.

Design diagrams and the project timeline further illustrate system workflow and development. In essence, this chapter establishes a clear and structured foundation for system implementation.

Chapter 4

Implementation Details

4.1 Overview

The proposed method to be implemented presents a unified and intelligent framework for healthcare data retrieval and real time clinical risk prediction using a Knowledge Graph driven architecture. Modern healthcare systems generate vast volumes of Electronic Health Records (EHR), but accessing and interpreting this data remains a challenge due to the complexity of structured query languages and fragmented storage systems. To address this limitation, the proposed system introduces a Natural Language Interface (NLI) that allows clinicians to interact with structured healthcare data using intuitive, queries that are comprehensible by humans.

The methodology integrates two complementary pipelines: (i) a semantic retrieval pipeline that converts natural language queries into SPARQL queries for structured data extraction, and (ii) a predictive analytics pipeline that performs real time risk assessment using temporal patient vital data. The integration of these pipelines ensures that the system not only retrieves relevant clinical insights efficiently but also supports proactive decision making by identifying potential health risks before they escalate.

Additionally, the framework emphasizes usability, scalability and safety by incorporating query validation mechanisms, controlled access to the Knowledge Graph and efficient data processing techniques. This combination transforms traditional data retrieval systems into intelligent clinical support tools.

4.2 Knowledge Graph Construction and Data Processing

The system begins with the construction of a healthcare Knowledge Graph using an Extract-Transform-Load (ETL) pipeline, which converts structured clinical data into a semantic representation suitable for advanced querying and analysis.

- **Data Extraction:** Clinical data is obtained in tabular format, containing patient demographics (age, gender, BMI), vital signs (heart rate, blood pressure, oxygen saturation), laboratory values, and hospital stay information such as admission and discharge details. This data may originate from synthetic datasets or real world EHR systems.
- **Data Cleaning and Normalization:** Before transformation, the dataset undergoes preprocessing steps including handling missing values, removing inconsistencies and normalizing numerical attributes. This ensures data quality and consistency for downstream processing.
- **Transformation:** The cleaned data is mapped into Resource Description Framework (RDF) triples, where each data point is represented as a subject-predicate-object relationship.
- **Ontology Mapping:** The transformation process aligns data attributes with a predefined healthcare ontology, ensuring semantic consistency across entities such as patients, diseases, medications, and observations.
- **Storage:** The generated RDF triples are stored in Turtle (.ttl) format, which provides a human-readable and machine-processable representation of graph data. These files are then loaded into a triple store such as Apache Jena Fuseki.
- **Temporal Modeling:** Each observation is associated with a timestamp, enabling the creation of a Temporal Knowledge Graph. This allows the system to capture dynamic changes in patient health over time, which is essential for predictive analytics.

The resulting Knowledge Graph provides a semantically rich and interconnected representation of healthcare data, enabling efficient querying and advanced analytical capabilities.

4.3 Natural Language to SPARQL Retrieval Pipeline

The retrieval pipeline enables clinicians to query the Knowledge Graph using natural language, eliminating the need for expertise in structured query languages.

- **Query Input and Preprocessing:** The process begins when a user submits a natural language query. The system performs preprocessing steps such as tokenization, lowercasing, and removal of stop words to standardize the input.
- **Named Entity Recognition (NER):** The system applies NER techniques to identify key entities such as diseases, symptoms, patient identifiers, and medical parameters within the query.
- **Entity Linking:** Extracted entities are matched to corresponding nodes in the Knowledge Graph using similarity matching and string comparison techniques. This step ensures that user input aligns with graph entities.
- **Context and Prompt Construction:** A structured prompt is generated by combining the query with Knowledge Graph schema information, ontology details and predefined examples. This context-rich prompt guides the Large Language Model in generating accurate queries.
- **SPARQL Query Generation:** A Large Language Model (Gemini) interprets the prompt and generates a SPARQL query that captures the intent of the user query.
- **Query Validation and Safety Enforcement:** To ensure secure execution, the generated query is validated to allow only read only operations (SELECT queries). Additionally, prefix constraints and LIMIT clauses are enforced to prevent excessive data retrieval and ensure system stability.
- **Execution and Result Formatting:** The validated query is executed against the triple store using a SPARQL endpoint. The results are retrieved in tabular JSON format, which can be further formatted into human-readable summaries for user interpretation.

This pipeline enables seamless and intuitive interaction with structured healthcare data, significantly improving accessibility for non-technical users.

4.4 Real Time Predictive Analytics Framework

The predictive analytics component enhances the system by enabling continuous monitoring and early detection of clinical risks based on temporal patient data.

4.4.1 Inference Pipeline

- **Trigger Mechanism:** The prediction process is event-driven and is triggered whenever new vital observations are added to the Knowledge Graph.
- **Feature Extraction:** A sliding temporal window (e.g., $t-2$ to t) is used to extract recent patient vitals, capturing short-term trends in physiological parameters.
- **Data Structuring:** The extracted data is organized into a structured tensor of size $(3, 6)$, where rows represent time steps and columns represent different vital features.
- **LSTM Model:** A Long Short Term Memory (LSTM) neural network with 64 hidden units is used to model temporal dependencies and detect patterns indicative of clinical deterioration.
- **Regularization:** A dropout layer with a rate of 0.2 is applied to prevent overfitting and improve generalization.
- **Classification Layer:** A Softmax activation function produces probability distributions across multiple risk categories.

4.4.2 Risk Prediction Outcomes

The system predicts several clinically significant risk conditions:

- **Sepsis Deterioration Risk:** Identifies early signs of systemic infection and rapid health decline.
- **Hypoxia Risk:** Detects low oxygen saturation levels that may indicate respiratory distress.
- **Hypertension Risk:** Identifies abnormal increases in blood pressure.
- **Fall Risk (Hypotension):** Detects sudden drops in blood pressure that may lead to fainting or falls.

4.4.3 Alert Generation and Notification

When predicted risk probabilities exceed predefined thresholds, the system generates alerts that include:

- Risk category and severity level
- Confidence score of prediction
- Timestamp of detection

These alerts are displayed to clinicians through the user interface thereby aimed at timely intervention and improved patient care.

4.5 Software and Hardware Requirements

4.5.1 Hardware Requirements

- **Processor:** Quad-core CPU (Intel i5 11th Gen or AMD Ryzen 7 5800X)
- **RAM:** 16GB
- **Operating System:** Ubuntu 22.04 LTS or Windows 10

4.5.2 Software Requirements

- **Programming Language:** Python 3.9+
- **LLM:** Gemini-2.5-flash for NL-to-SPARQL generation
- **Orchestration:** LangChain for agent workflow management
- **Triple Store:** Apache Jena Fuseki for Knowledge Graph storage
- **Deep Learning Framework:** TensorFlow and Keras for LSTM model
- **IDE:** Visual Studio Code

4.5.3 Dataset

- **Data Fields:** Age, gender, BMI, medical history (diabetes, hypertension), vital signs (heart rate, SpO2, systolic BP, diastolic BP, temperature, respiratory rate), hospital stay details, and clinical metrics (glucose, creatinine, cholesterol).
- **Number of Entries:** Approximately 6,000 labeled entries (Stable: 2,902; Sepsis: 1,196; Hypertension Crisis: 888; Hypoxia: 596; Fall Risk: 418).
- **Description:** A Synthetic Clinical Tabular Dataset sourced from Kaggle is used to construct the Knowledge Graph via an ETL pipeline, enabling representation of patient histories and temporal vital trends.

4.6 Summary of the chapter

This chapter presented the implementation details of the system, integrating natural language query processing with Knowledge Graph based data retrieval and real time clinical risk prediction. The system combines semantic querying through an NLto SPARQL pipeline with an LSTM based predictive framework for continuous patient monitoring. The Knowledge Graph enables structured and efficient data access, while the predictive module facilitates early detection of clinical risks. The chapter also outlined the hardware and software requirements necessary for implementation and also provides a scalable, intelligent, and user friendly solution for healthcare data analysis and decision support.

Chapter 5

Results and Discussions

5.1 Overview

This chapter presents the results obtained from the implementation of the proposed system, which integrates natural language querying with clinical risk prediction capabilities. The system enables users to retrieve structured medical information using natural language queries while also analyzing patient vital sign data to detect potential health risks.

The first part of the results evaluates the performance of the natural language to SPARQL conversion module. This component allows users to submit queries in natural language, which are then interpreted by the LLM based agent and translated into SPARQL queries for execution on the healthcare knowledge graph. The outputs demonstrate the system’s ability to correctly interpret user intent, generate valid SPARQL queries, and retrieve relevant medical information from the knowledge graph.

The second part of the results focuses on the clinical risk classification module. Using patient vital parameters such as heart rate, blood pressure, oxygen saturation (spO2) and other physiological indicators, the trained machine learning model predicts possible risk conditions including Fall Risk, Hypertension Crisis, Hypoxia, Sepsis Deterioration and Stable states. The effectiveness of the prediction model is evaluated using standard performance metrics such as accuracy, precision, recall, and F1-score.

In addition to overall model performance, class-wise evaluation and confusion matrix analysis are presented to examine how accurately the model identifies each medical condition.

5.2 UI Design

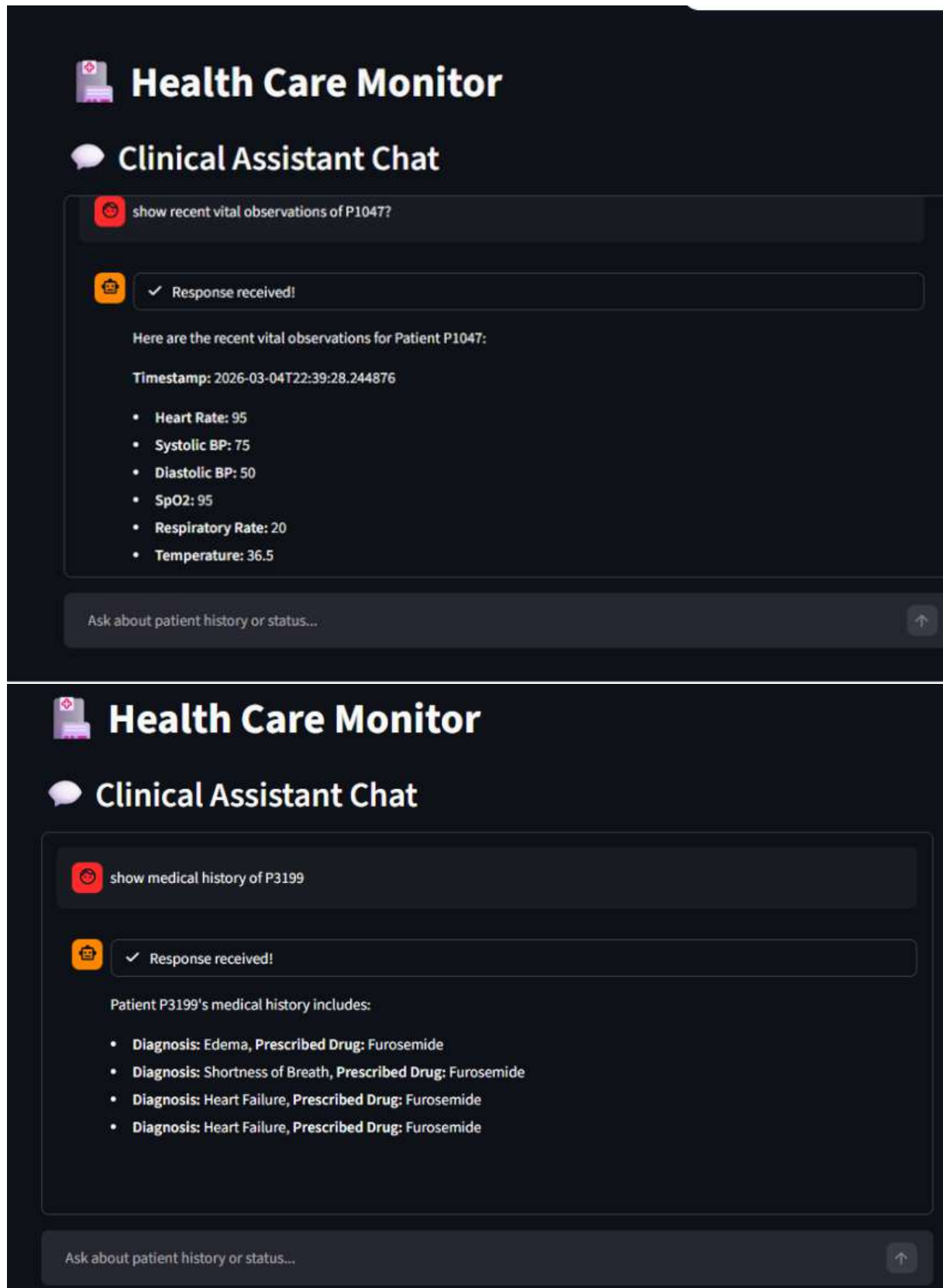


Figure 5.1: NL to SPARQL query result

 **AI Assistant Context**

Monitoring Patient: P1047

Active Stay ID: Stay_c28ced

 **Real-Time Vitals Entry**

Heart Rate

75

-

+

Systolic BP

115

-

+

Diastolic BP

75

-

+

Resp Rate

16

-

+

Temp (°C)

37.00

-

+

SpO2 (%)

98

-

+

LOG VITALS & RUN AI

☒ Vitals logged at simulated time: 2026-03-04T10:39:28.244876

AI requires 3 data points to predict. Collecting data...

Enter Patient ID:

P1047

Re-Admit / Start New Stay

 **AI Assistant Context**

Monitoring Patient: P1047

Active Stay ID: Stay_c28ced

 **Real-Time Vitals Entry**

Heart Rate

78

-

+

Systolic BP

98

-

+

Diastolic BP

65

-

+

Resp Rate

16

-

+

Temp (°C)

37.00

-

+

SpO2 (%)

98

-

+

LOG VITALS & RUN AI

☒ Vitals logged at simulated time: 2026-03-04T14:39:28.244876

AI requires 3 data points to predict. Collecting data...

Figure 5.2: Vital entry into the KG



AI Assistant Context

Monitoring Patient: P1047

Active Stay ID: Stay_c28ced



Real-Time Vitals Entry

Heart Rate	Systolic BP	Diastolic BP
82 - +	85 - +	55 - +
Resp Rate	Temp (°C)	SpO2 (%)
18 - +	36.80 - +	97 - +

LOG VITALS & RUN AI

☒ Vitals logged at simulated time: 2026-03-04T18:39:28.244876

☒ Patient is Stable (Confidence: 94.6%).

Heart Rate	Systolic BP	Diastolic BP
95 - +	132 - +	86 - +
Resp Rate	Temp (°C)	SpO2 (%)
30 - +	39.00 - +	85 - +

LOG VITALS & RUN AI

☒ Vitals logged at simulated time: 2026-03-03T20:47:12.087082

 **CRITICAL ALERT: Hypoxia DETECTED!** (Confidence: 95.1%)

Heart Rate	Systolic BP	Diastolic BP
95 - +	75 - +	50 - +
Resp Rate	Temp (°C)	SpO2 (%)
20 - +	36.50 - +	95 - +

LOG VITALS & RUN AI

☒ Vitals logged at simulated time: 2026-03-03T20:52:57.890356

 **CRITICAL ALERT: Fall Risk DETECTED!** (Confidence: 96.2%)

Figure 5.3: Risk Alert

5.3 Evaluation Tables and Metrics

Table 5.1: Overall performance metrics of the classification model

Metric	Value
Accuracy	0.92
Macro Precision	0.90
Macro Recall	0.90
Macro F1-score	0.90
Weighted F1-score	0.91

Table 5.1 presents the overall performance metrics of the classification model where the model achieves an accuracy of 0.92, indicating strong predictive capability. The macro precision, recall, and F1-score are all approximately 0.90, demonstrating balanced performance across all classes. Additionally, the weighted F1-score of 0.91 confirms that the model maintains consistency even when class distribution is considered.

Table 5.2: Class-wise performance metrics

Class	Precision	Recall	F1-score	Support
Fall Risk	0.86	0.97	0.91	77
Hypertension Crisis	0.90	0.87	0.88	108
Hypoxia	0.86	0.77	0.81	103
Sepsis Deterioration	0.99	0.99	0.99	203
Stable	0.90	0.92	0.91	309

The class-wise evaluation shown in Table 5.2 highlights the model’s effectiveness across different clinical conditions. The model achieves the greatest results for Sepsis Deterioration with a precision and recall of 0.99. Fall Risk shows high recall (0.97) but slightly lower precision (0.86), indicating occasional false positives. Hypoxia has comparatively lower recall (0.77), suggesting some missed detections, while Stable cases maintain a balanced performance with an F1-score of 0.91.

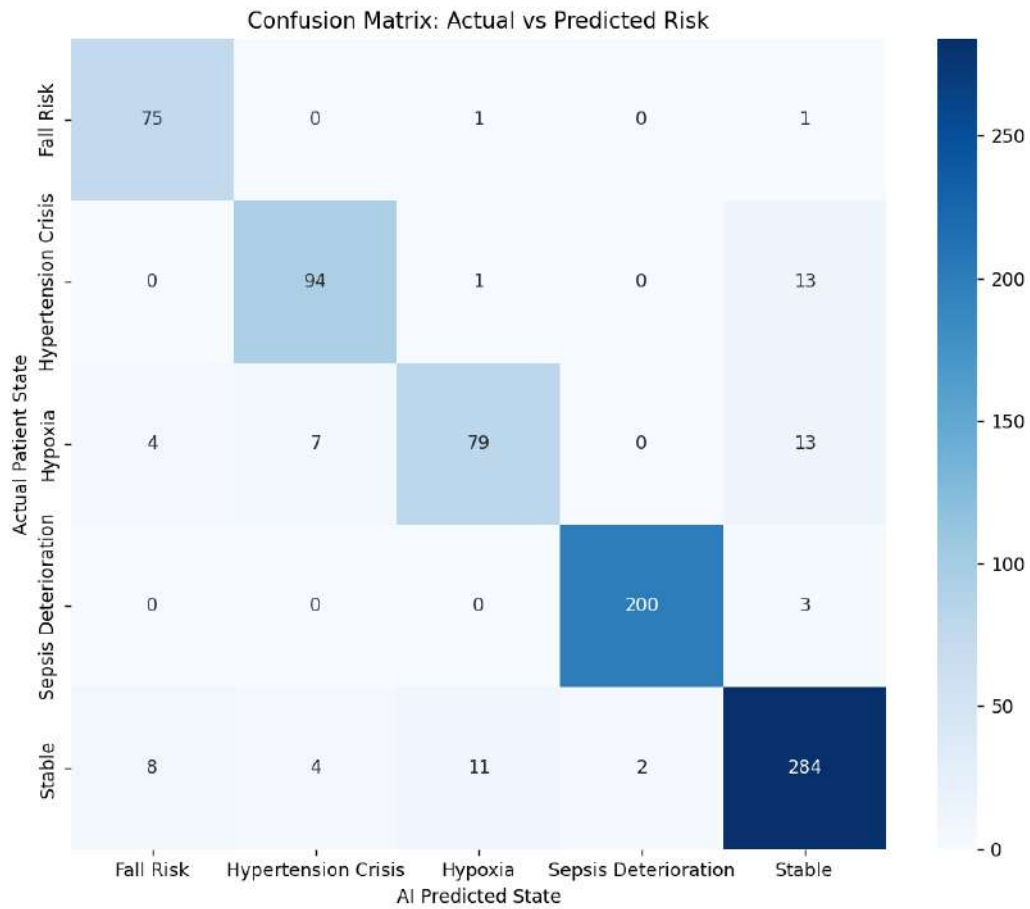


Figure 5.4: Confusion matrix of medical risk classification

Figure 5.4 illustrates the distribution of correct predictions where Sepsis Deterioration records the highest correct predictions (200 cases), followed by Stable (284 cases). Hypoxia and Hypertension Crisis show slightly lower correct classifications with 79 and 94 cases.

Table 5.3: Notable misclassification summary

Actual Class	Misclassification
Hypoxia	Predicted as Stable (13)
Hypertension Crisis	Predicted as Stable (13)
Stable	Predicted as other classes (25 total)

The misclassification trends in Table 5.4 reveal that Hypoxia and Hypertension Crisis are occasionally predicted as Stable (13 instances each), suggesting overlapping physio-

logical patterns. Additionally, Stable cases are misclassified into other categories in 25 instances, highlighting the challenge of distinguishing borderline conditions.

Table 5.4: Example prediction cases

Case Type	Vital Pattern	Predicted Class	Result
Success	High HR + Low BP	Sepsis Deterioration	Correct
Success	Low BP	Fall Risk	Correct
Borderline	Slightly low SpO ₂	Stable	Incorrect
Borderline	Mild hypertension	Stable	Incorrect

5.4 Discussion

The evaluation results demonstrate that the proposed clinical risk classification model performs effectively in identifying multiple patient risk conditions using vital sign parameters. The overall accuracy of 0.92 indicates that the model is capable of correctly classifying the majority of patient cases across the five risk categories. In addition to accuracy, the macro precision, recall, and F1-score values are all approximately 0.90, indicating balanced performance across different classes without significant bias toward any particular category. The weighted F1-score of 0.91 further confirms that the classifier maintains strong predictive capability even when class distribution differences are taken into account.

The class wise evaluation provides deeper insight into how the model performs across individual medical risk categories. The results show that the model achieves exceptionally high performance for the *Sepsis Deterioration* class, with both precision and recall values reaching 0.99. This shows that the model is highly reliable in detecting severe clinical deterioration conditions, which is particularly important in healthcare monitoring systems where early detection of critical states can significantly improve patient outcomes.

Similarly, the *Fall Risk* category demonstrates strong recall performance (0.97), suggesting that the model is highly effective at identifying patients who may be at risk of falls. This high recall reduces the probability of missing potential fall risk cases, which is crucial for preventive care in clinical environments. The *Hypertension Crisis* category also exhibits stable performance with balanced precision and recall values of approximately

0.90 and 0.87 respectively.

The model performs moderately well in detecting *Hypoxia* cases, achieving an F1-score of 0.81. While this still represents acceptable performance, it is comparatively lower than other classes. The confusion matrix analysis reveals that a portion of hypoxia cases are misclassified as *Stable*. This may occur in borderline scenarios where oxygen saturation levels are close to normal thresholds, making it more difficult for the model to distinguish between stable and early hypoxia conditions.

The confusion matrix visualization further supports the overall robustness of the model. The strong diagonal dominance indicates that the majority of predictions match the true class labels. Correct prediction counts across different classes remain high, particularly for *Stable* and *Sepsis Deterioration* categories. However, some misclassification patterns are observed, primarily between physiologically related conditions. For example, both *Hypoxia* and *Hypertension Crisis* occasionally appear as *Stable* predictions. This behavior reflects the inherent overlap in physiological patterns that can occur in real-world patient monitoring data.

The example prediction cases further illustrate how the model behaves under different vital sign conditions. Cases with clear physiological indicators, such as high heart rate combined with low blood pressure, are accurately classified as *Sepsis Deterioration*. Similarly, scenarios involving low blood pressure are successfully identified as *Fall Risk*. However, borderline cases involving slightly reduced oxygen saturation or mild hypertension may sometimes be interpreted as stable conditions. These observations highlight the importance of threshold sensitivity in clinical classification models.

Despite these minor limitations, the results indicate that the model maintains consistent predictive performance across multiple risk categories. The combination of high accuracy, balanced precision and recall, and strong detection of critical deterioration states demonstrates that the approach is suitable for supporting clinical monitoring systems. The model’s interpretable decision logic also aligns with medically relevant physiological thresholds, making it easier for healthcare practitioners to understand the basis of the predictions.

Overall, the results suggest that the proposed system provides reliable multi-class risk classification while maintaining robustness across different clinical conditions. With further enhancements such as adaptive threshold adjustment and integration of temporal

patient monitoring data, the model could be extended to provide even more accurate and context-aware risk predictions in real-world healthcare environments.

5.5 Summary of the chapter

This chapter presented the results obtained from the implementation of the proposed healthcare data analysis system, focusing on both natural language query processing and clinical risk classification. The evaluation of the NL-to-SPARQL module demonstrated the system’s ability to accurately interpret user queries and retrieve relevant information from the knowledge graph in a structured and efficient manner.

The performance of the clinical risk prediction model was analyzed using standard evaluation metrics, including accuracy, precision, recall, and F1-score. The results indicated strong overall performance, with particularly high accuracy in detecting critical conditions such as sepsis deterioration. Class wise analysis and confusion matrix evaluation further highlighted the model’s effectiveness across multiple risk categories, while also identifying minor misclassification patterns in borderline cases.

Example prediction scenarios illustrated the practical behavior of the system in real world situations, demonstrating its capability to interpret patient vital data and generate meaningful risk assessments. Despite some limitations, such as reduced sensitivity near classification thresholds, the model maintained balanced and reliable performance.

Overall, the findings confirm that the proposed system effectively integrates natural language querying with clinical risk prediction, providing a robust framework for healthcare data retrieval and patient monitoring. The results validate the system’s potential to support informed decision-making in clinical environments.

Chapter 6

Conclusion

6.1 Conclusion

This project showcases the creation of an AI driven clinical decision support system that combines natural language processing, knowledge graph technology, and predictive analytics to improve healthcare data access and real-time risk monitoring. It allows clinicians to interact with structured Electronic Health Records using easy natural language queries. This removes the need for technical skills to retrieve data and ensures quick and accurate access to essential patient information.

The proposed framework includes a secure NL to SPARQL agent, a detailed healthcare knowledge graph and an LSTM based prediction model. This combination offers both historical insights and proactive risk assessment. The system effectively identifies conditions like sepsis deterioration, hypoxia, hypertension crisis, and fall risk. It also ensures safe querying through policy driven execution methods. Additionally, real time alert features help healthcare professionals respond quickly by notifying them of potential risks based on changing patient vitals. By bridging the gap between complex healthcare data and user-friendly interaction, the system changes traditional data retrieval into an intelligent and accessible process. Its modular and scalable design allows it to integrate smoothly with current healthcare systems, making it suitable for real-world use.

Overall, this work shows the promise of combining artificial intelligence, knowledge graphs and clear query methods to facilitate data driven clinical decisions. With further improvements like large scale validation, integration of various data types, and deployment in clinical settings, the proposed system can greatly enhance patient care, operational efficiency, and timely medical intervention in today's healthcare ecosystem.

6.2 Future Scope

- Extension to multi-hospital datasets using privacy-preserving techniques such as federated learning, enabling the system to train personalized risk prediction models without sharing sensitive patient data.
- Integration with real-time wearable devices to continuously stream vital parameters such as heart rate and oxygen saturation into the temporal knowledge graph, allowing dynamic monitoring and early detection of patient deterioration using LSTM-based models.
- Incorporation of multimodal data sources including medical imaging (e.g., MRI and CT scans) processed using advanced deep learning models such as Vision Transformers, enabling richer clinical context through combined graph and visual representations.
- Enhancement of explainability using advanced Explainable AI techniques such as SHAP, allowing visualization of key factors and temporal patterns that contribute to risk predictions, thereby improving clinician trust and interpretability.
- Development of patient-facing conversational agents that enable both clinicians and patients to interact with the system using voice or chat interfaces, providing personalized insights while maintaining strict query safety and knowledge graph grounding.
- Expansion of the knowledge graph to include larger, real-world electronic health record datasets, improving system robustness and enabling more comprehensive clinical decision support.
- Deployment as an integrated Clinical Decision Support System (CDSS) within hospital infrastructures, enabling seamless interaction with existing healthcare information systems and workflows.

6.3 Summary of the chapter

This chapter summarizes the outcomes of the proposed NLP-enabled healthcare data analysis system, which integrates natural language query processing, knowledge graph-based

data retrieval, and clinical risk prediction. The system demonstrates how users can access structured healthcare information through natural language queries while supporting real-time monitoring and prediction of patient risk conditions using machine learning models.

The results highlight strong predictive performance with high accuracy and balanced evaluation metrics across multiple risk categories. The system also enhances usability by enabling efficient data retrieval and generating alerts for early detection of patient deterioration, making it a practical decision- support tool for healthcare professionals.

Furthermore, potential future enhancements such as integration with multi- hospital datasets, real-time wearable data, and multimodal analysis using advanced AI techniques are discussed. Overall, the proposed system effectively bridges the gap between complex healthcare data and end-users, supporting faster and more informed clinical decision-making.

References

- [1] Liang, S., Stockinger, K., de Farias, T.M., Anisimova, M., and Gil, M. (2021) ‘Querying knowledge graphs in natural language’, *Journal of Big Data*, 8(3), pp. 1–23.
- [2] Pan, Y., Wang, M., Lu, L., Lamsal, R., Pärn, E., Zlatanova, S., and Brilakis, I. (2026) ‘LLM-enabled multi-agent framework for natural language interaction with graph-based digital twins’, *Automation in Construction*, 183, p. 106791.
- [3] D’Abramo, J., Zugarini, A., and Torroni, P. (2024) ‘Investigating Large Language Models for Text-to-SPARQL Generation’, *Proceedings of the Association for Computational Linguistics*.
- [4] Chaturvedi, R. and Di Eugenio, B. (2024) ‘Temporal knowledge graph extraction and modeling across multiple documents for health risk prediction’, *Companion Proceedings of the ACM Web Conference 2024*, pp. 1181–1185.
- [5] Zhao, X., Liu, S., Yang, S.-Y., and Miao, C. (2025) ‘MedRAG: Enhancing Retrieval-augmented Generation with Knowledge Graph-Elicited Reasoning for Healthcare Copilot’, *Proceedings of the ACM Web Conference 2025*.
- [6] Liu, L.J., Ortiz-Soriano, V., Neyra, J.A., and Chen, J. (2022) ‘KIT-LSTM: Knowledge-guided Time-aware LSTM for Continuous Clinical Risk Prediction’, *2022 IEEE International Conference on Bioinformatics and Biomedicine (BIBM)*, pp. 1–8.
- [7] Park, J., Cho, Y., Lee, H., Choo, J., and Choi, E. (2021) ‘Knowledge Graph-based Question Answering with Electronic Health Records’, *Proceedings of Machine Learning Research*, 149, pp. 1–17.
- [8] Postiglione, M., Bean, D., Kraljevic, Z., Dobson, R.J.B., and Moscato, V. (2024) ‘Predicting Future Disorders via Temporal Knowledge Graphs and Medical Ontologies’, *IEEE Journal of Biomedical and Health Informatics*, 28(7), pp. 4238–4249.

- [9] Xie, A. (2026) ‘Deep Representation Learning for Risk Prediction in Electronic Health Records Using Self-Supervised Methods’, *Preprints.org*.
- [10] Shaik, S., Kanakam, P., Hussain, S.M., and Suryanarayana, D. (2016) ‘Transforming natural language query to SPARQL for semantic information retrieval’, *International Journal of Engineering Trends and Technology (IJETT)*, 41(7), pp. 347–349.
- [11] Yang, M., Peng, Z., van Pul, C., Andriessen, P., Dong, K., Silvertand, D., Li, J., Liu, C., and Long, X. (2024) ‘Continuous prediction and clinical alarm management of late-onset sepsis in preterm infants using vital signs from a patient monitor’, *Computer Methods and Programs in Biomedicine*, 255, p. 108335.

Appendix A: Presentation

In collaboration with



STRUCTURED HEALTHCARE DATA RETRIEVAL VIA NATURAL LANGUAGE AND PREDICTIVE ANALYSIS

Project Guide

Dr. Amitha Mathew

RENO JACOB ZACHARIAH (U2203176)

REUBEN MATHEW SHAJI (U2203177)

ROHAN CIJOY (U2203181)

VYSHNAV A K (U2203214)

TABLE OF CONTENTS

- | | |
|--|---|
| 1. Problem Definition | 9. Result and Output |
| 2. Problem Objective | 10. Assumptions |
| 3. Purpose and Need | 11. Hardware and Software Requirements |
| 4. Literature Survey | 12. Work Breakdown and Responsibilities |
| 5. Proposed Methods | 13. Gantt Chart |
| 6. Overall Architecture Diagram | 14. Risks and Challenges |
| 7. Module wise diagrams and Descriptions | 15. Conclusion |
| 8. KG Schema Designs | 16. References |

PROBLEM DEFINITION

Healthcare professionals lack an integrated system that allows for intuitive, natural language querying of Electronic Health Record (EHR) data forcing a reliance on IT specialists leaving critical, time-sensitive insights undiscovered within the data. A predictive system is also generated in such a way that it enables clinicians to not only ask questions in plain language but also receive forecasted risk predictions derived from patient history and vitals.

3

PROBLEM OBJECTIVE

- To propose an NLP-based natural language interface for querying healthcare databases.
- To convert everyday user queries into structured SPARQL commands automatically.
- To provide clinicians with a reliable, accurate, and user-friendly way to access healthcare information.
- Finally, to build a predictive Knowledge Graph and implement prediction based on it on a time window

4

PURPOSE AND NEED

- A system for clinicians lacking technical knowledge to query in natural language is currently absent.
- To convert everyday user queries into structured SPARQL commands automatically.
- To provide clinicians with a reliable, accurate, and user-friendly way to access healthcare information.
- To build a predictive Knowledge Graph and implement prediction based on it on a time window
- To alert the clinician on a predicted risk of a patient to initiate mitigation beforehand

5

LITERATURE SURVEY

1. BASE PAPER: "QUERYING KNOWLEDGE GRAPHS IN NATURAL LANGUAGE" (LIANG ET AL., 2021)

- The system translates natural language to SPARQL through a five-step modular pipeline: question analysis, type classification, phrase mapping, query generation, and ranking.
- A primary advantage is the separation of knowledge graph-dependent and independent modules, allowing for easy adaptation to new data domains without re-training core components.
- A key limitation is the "phrase mapping bottleneck," where the system fails to generate queries if necessary properties are only implicitly stated in the input question.
- Low recall persists for "Boolean" and complex "Count" queries due to structural complexity and a lack of diverse training data for these specific types

6

LITERATURE SURVEY

2. "TEMPORAL KNOWLEDGE GRAPH EXTRACTION AND MODELING FOR HEALTH RISK PREDICTION"

- The proposed method uses a span-based joint entity-relation extraction framework with pre-trained transformers to extract patient-centric temporal knowledge graphs from clinical text.
- A primary advantage is the ability to fuse temporal orderings within single documents and across multiple patient visits to model longitudinal disease progression.
- Incorporating dynamic Graph Neural Networks (GNNs) allows the model to capture evolving entity properties and relationships as a patient's health status changes over time.
- A key limitation is the potential for "distribution shift" when transferring models between different hospital datasets due to variations in note-taking styles and EHR systems.

7

LITERATURE SURVEY

3. "TRANSFORMING NATURAL LANGUAGE QUERY TO SPARQL FOR SEMANTIC INFORMATION RETRIEVAL"

- The system utilizes the LODQA (Linked Open Data Question Answering) architecture, which parses NL queries into pseudo graph patterns (PGP) before grounding them in target datasets.
- A primary advantage is that it allows non-technical end-users to search complex RDF data without needing to learn the formal syntax of the SPARQL language.
- The method produces more efficient and precise semantic results (triplet/tabular form) compared to traditional search engines that return a large volume of non-specific links.
- A key limitation is the difficulty in representing highly complex or deeply nested queries, as machines often struggle to interpret the nuances of human speech.

8

LITERATURE SURVEY

4. "PREDICTING FUTURE DISORDERS VIA TEMPORAL KNOWLEDGE GRAPHS AND MEDICAL ONTOLOGIES"

- The MedTKG framework models patient medical histories as chronological snapshots and predicts future disorders by identifying missing objects in quadruples.
- A primary advantage is integrating static medical ontologies (e.g., SNOMED CT) with dynamic clinical data to improve prediction accuracy. .
- A key limitation is low recall, caused by the extensive number of potential medical concepts and the complexity of predicting future events.
- Performance is sensitive to concept frequency, making the model less effective for rare diseases with low data support

9

LITERATURE SURVEY

5. "KNOWLEDGE GRAPH-BASED QUESTION ANSWERING WITH ELECTRONIC HEALTH RECORDS"

- The proposed method converts natural language queries into SPARQL by representing Electronic Health Records (EHR) as directed acyclic graphs rather than traditional relational tables.
- Graph-based approach naturally captures complex relations between entities and values, avoiding the performance-heavy "JOIN" operations required by SQL.
- A key limitation is the reliance on a predefined schema; if a user's question uses terms not explicitly mapped in the knowledge graph, the translation to SPARQL may fail.
- Low recall occurs for highly complex queries where the sequence of relations in the resulting SPARQL query becomes deeply nested or spans many "hops."

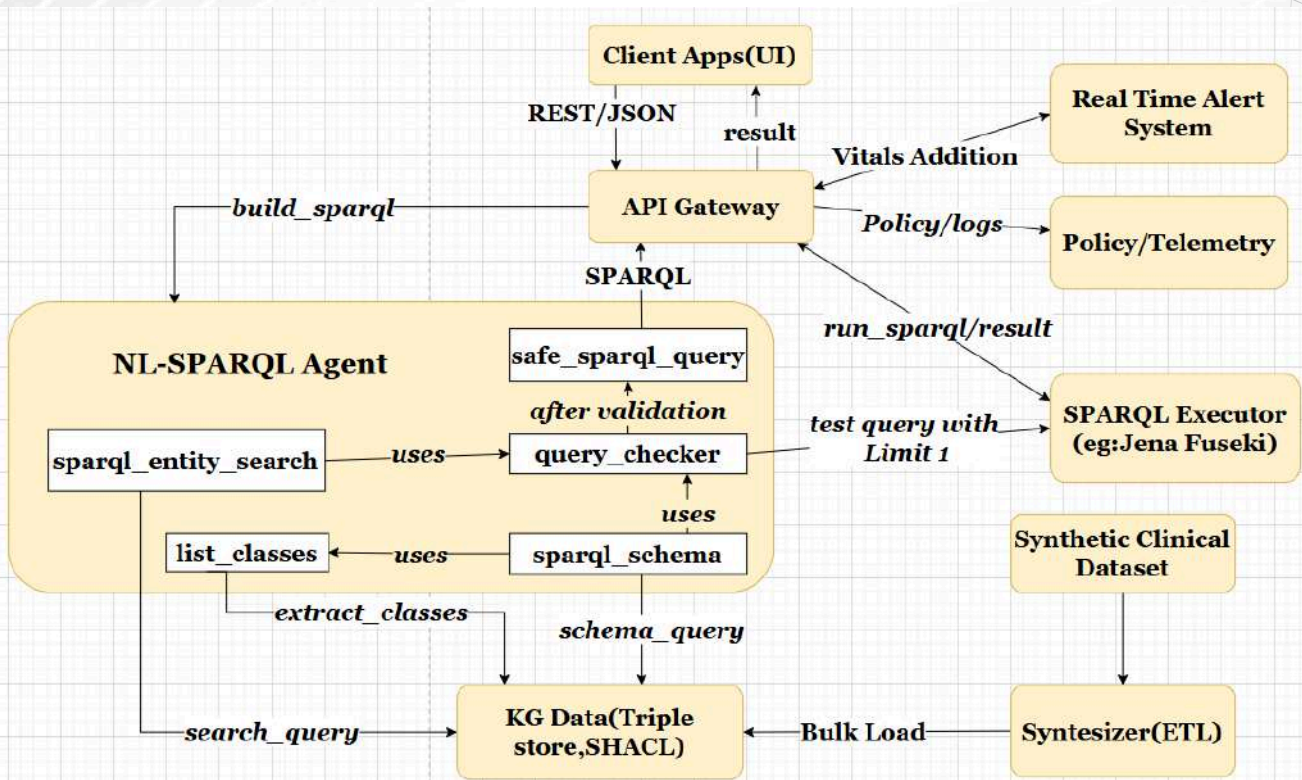
10

PROPOSED METHOD

- The system uses **natural language** to translate and create **SPARQL** queries over a **temporal knowledge graph**.
- **Knowledge graphs (KGs)** represent the complex relations between **medical entities** and their values more naturally than traditional relational tables and SPARQL is the specialized query language designed to navigate and retrieve information from these specific graph structures.
- The system also has a **KG feature extraction** and an **ML prediction** of outcomes like critical risk or episodes based on temporal vital patterns during a patient's stay in the hospital, along with medical context.
- A **sliding window LSTM** prediction engine operating on a Temporal Knowledge Graph is used to facilitate the predictive analysis.

11

OVERALL SYSTEM ARCHITECTURE

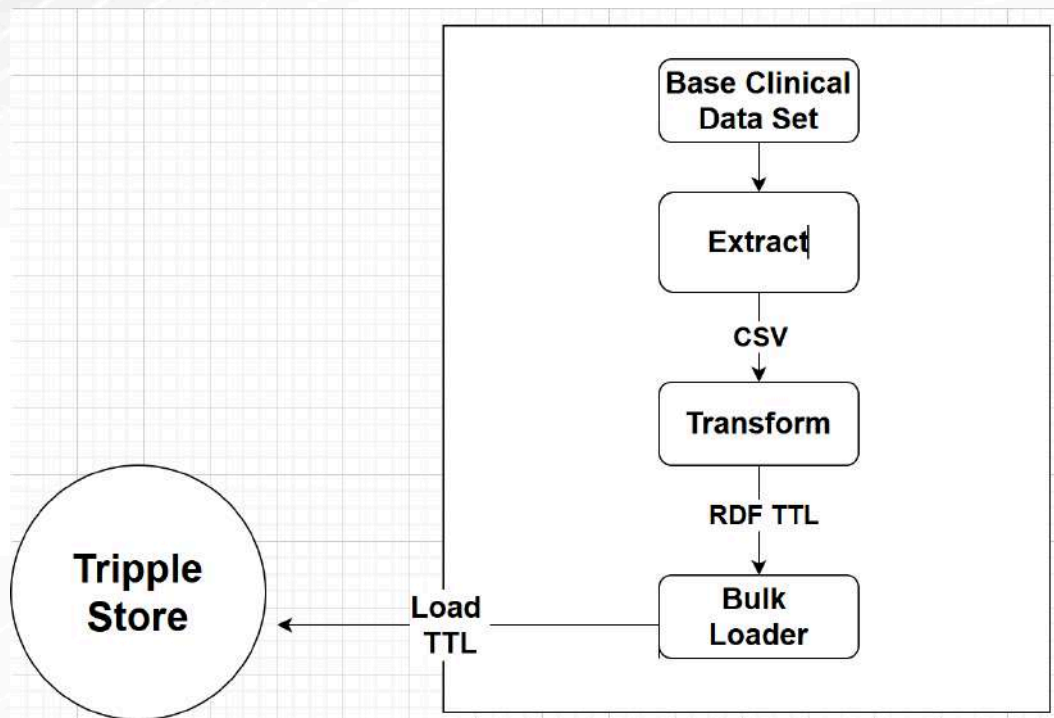


12

MODULE VISE DIAGRAMS & DESCRIPTONS

12

SYNTHESIZER



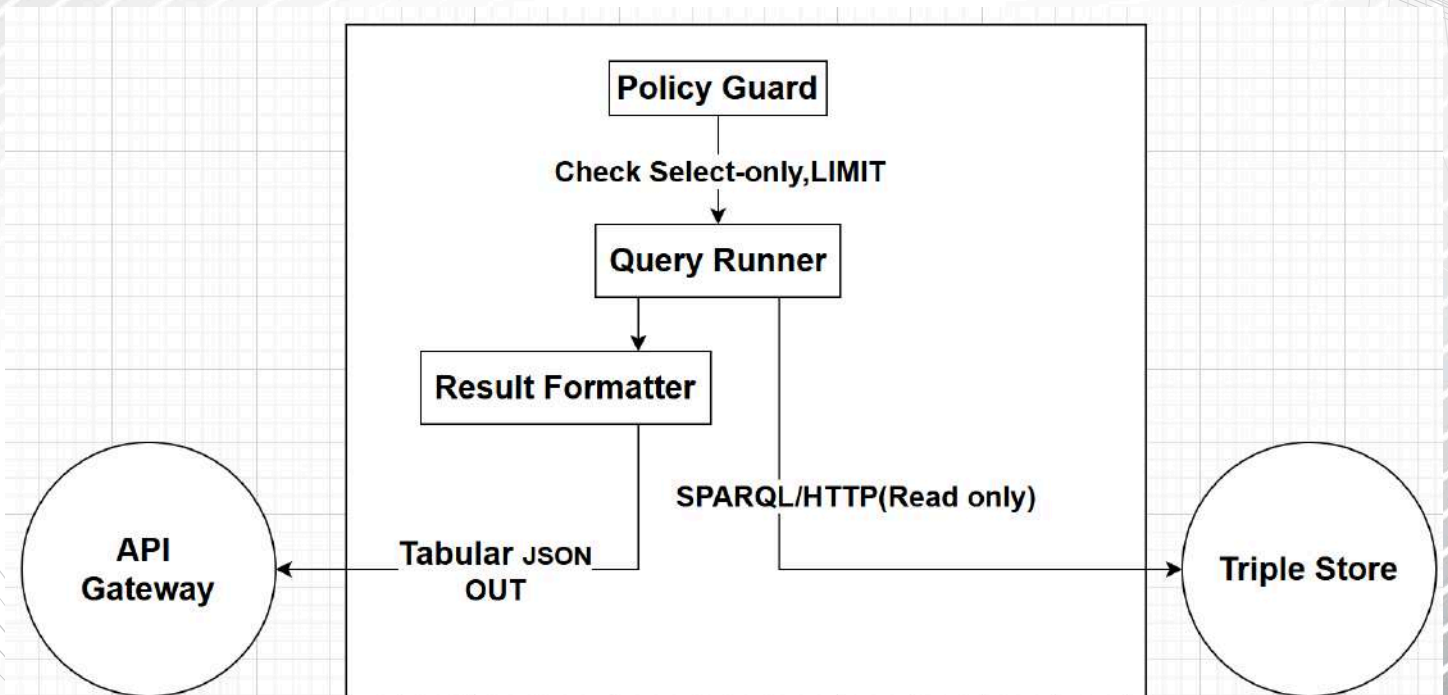
13

Synthesizer

- Takes the base clinical Dataset as a base for creating the extended version of Hospital data
- Generate csv's for patient data, hospital stay, vitals derived from the base dataset.
- Consists of Extract-Transform-Load (ETL) Pipeline which takes csv's and converts them into rdf format and saves them as .ttl(Turtle) files
- Using the csv's ,we synthesize ttl files containing rdf triples for patient, hospital stays, vitals observations during stay, doctors and hospital files.
- These ttl files are loaded into triple store for querying

14

SPARQL EXECUTOR



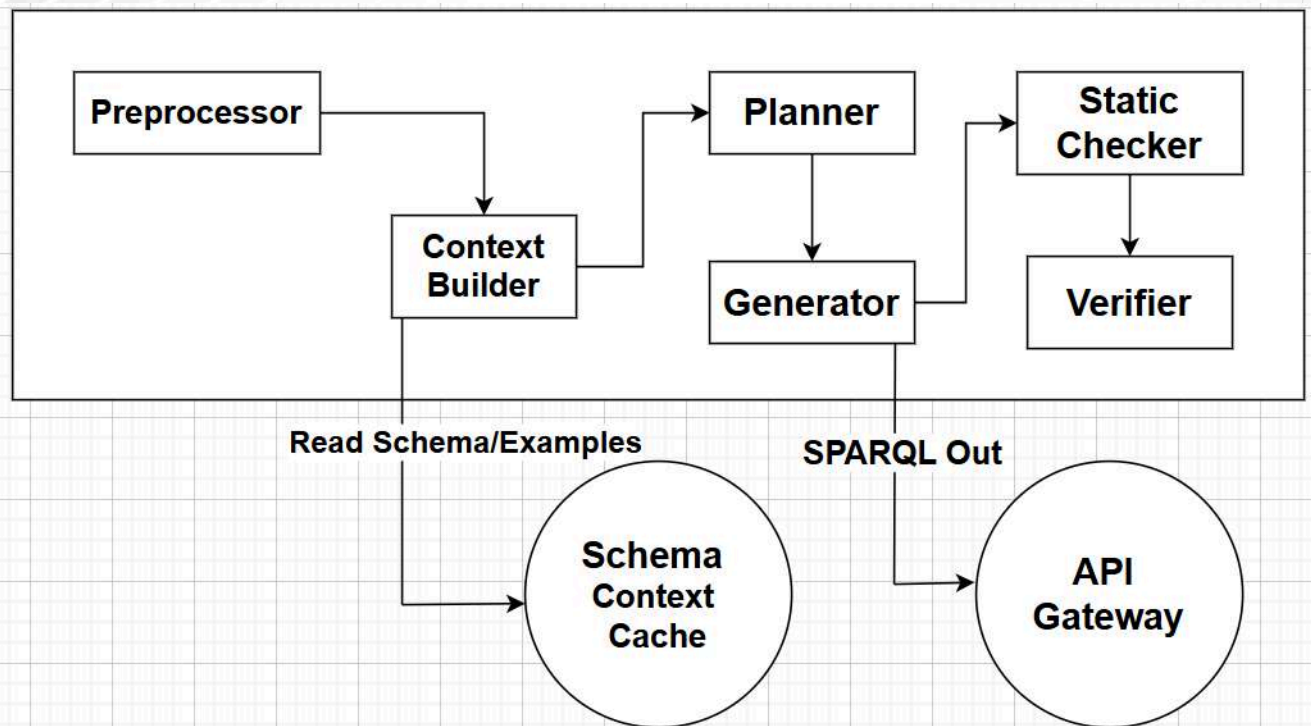
15

SPARQL Executor Module

- SPARQL Executor module runs queries safely on the KG for NL questions.
- Checks queries for SELECT-only and adds LIMITs to block harmful changes and limit data overload in read-only triple store.
- Executes SPARQL against GraphDB/Fuseki.
- Returns results as easy tabular JSON for agent and client apps.

16

NL-SPARQL AGENT



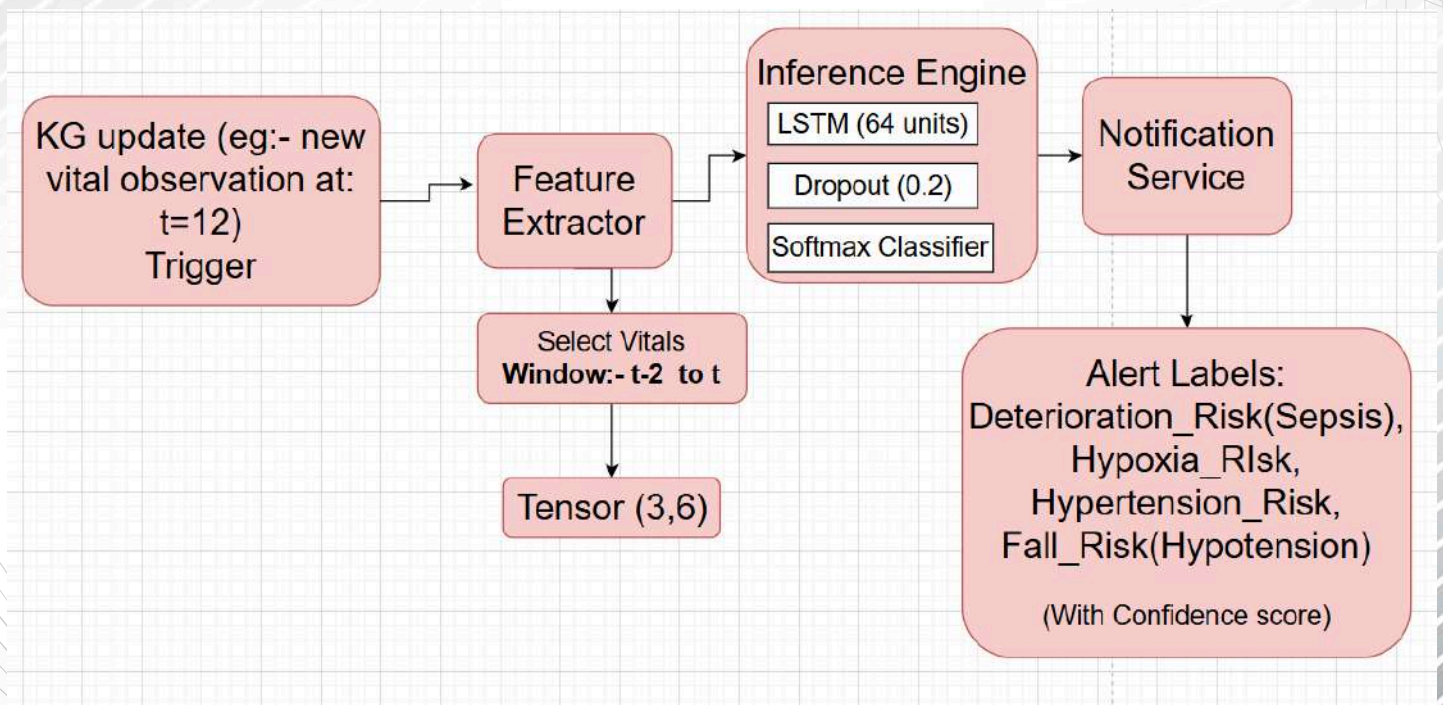
17

NL-SPARQL Agent Module

- Consist of 5 tools - safe_sparql_query, query_checker, sparql_schema, sparql_entity_search, list_classes.
- Pre-processor consists of two tools - list_classes which provides the classes of the KG & sparql_schema which provides schema of KG.
- Context builder uses these tools along with schema, instructions, few shot NL-SPARQL examples in prompt to plan the query.
- The agent generates the SPARQL.
- The static checker and verifier uses the query_checker tool and safe_sparql_query tool to validate read only queries are generated, with proper limits and prefixes.

18

PREDICTIVE ANALYTICS- ALERT SYSTEM



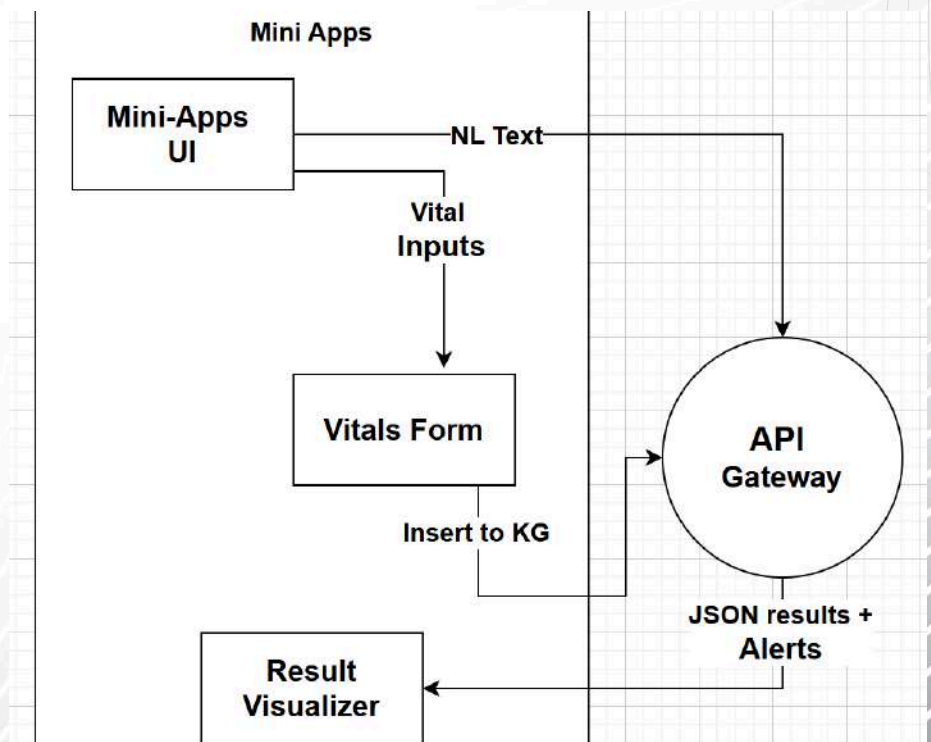
19

Predictive Analytics and Alert System

- Trigger: Event-driven activation via Knowledge Graph (KG) updates (e.g., new vital observation at $t=12$).
- Feature Extraction: Converts vitals from a temporal window ($t-2$ to t) into a structured (3, 6) Tensor.
- Inference Engine:
 - Architecture: LSTM (64 units) for temporal pattern recognition.
 - Optimization: Dropout (0.2) layer for regularization and overfit prevention.
 - Classification: Softmax Classifier produces probabilistic risk distributions.
- Notification: Dispatches alerts for Sepsis, Hypoxia, Hypertension, and Fall Risk with associated Confidence Scores.

20

MINI APPS



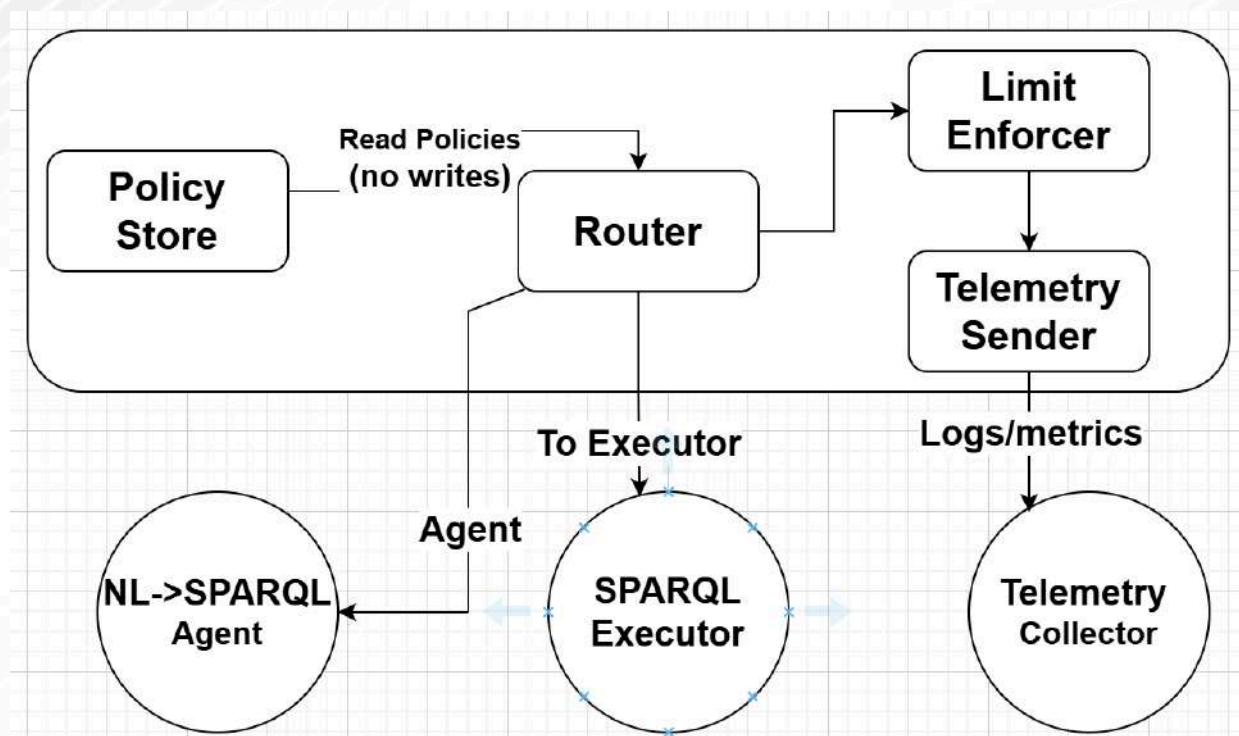
21

Mini Apps (UI) Module

- User interface consist of two parts : 1) NL to SPARQL Agent chat interface
2) Vitals Addition and alert system
- The chat interface allows users to ask NL questions and provides the final answer from NL-to-SPARQL agent.
- The alert system consist of form where you can admit a patient (creating a new stay) and then add vitals which will be timestamped with 4 hours gap.
- And notification is shown when LSTM predictor is triggered, showing risk_state with confidence.

22

API GATEWAY



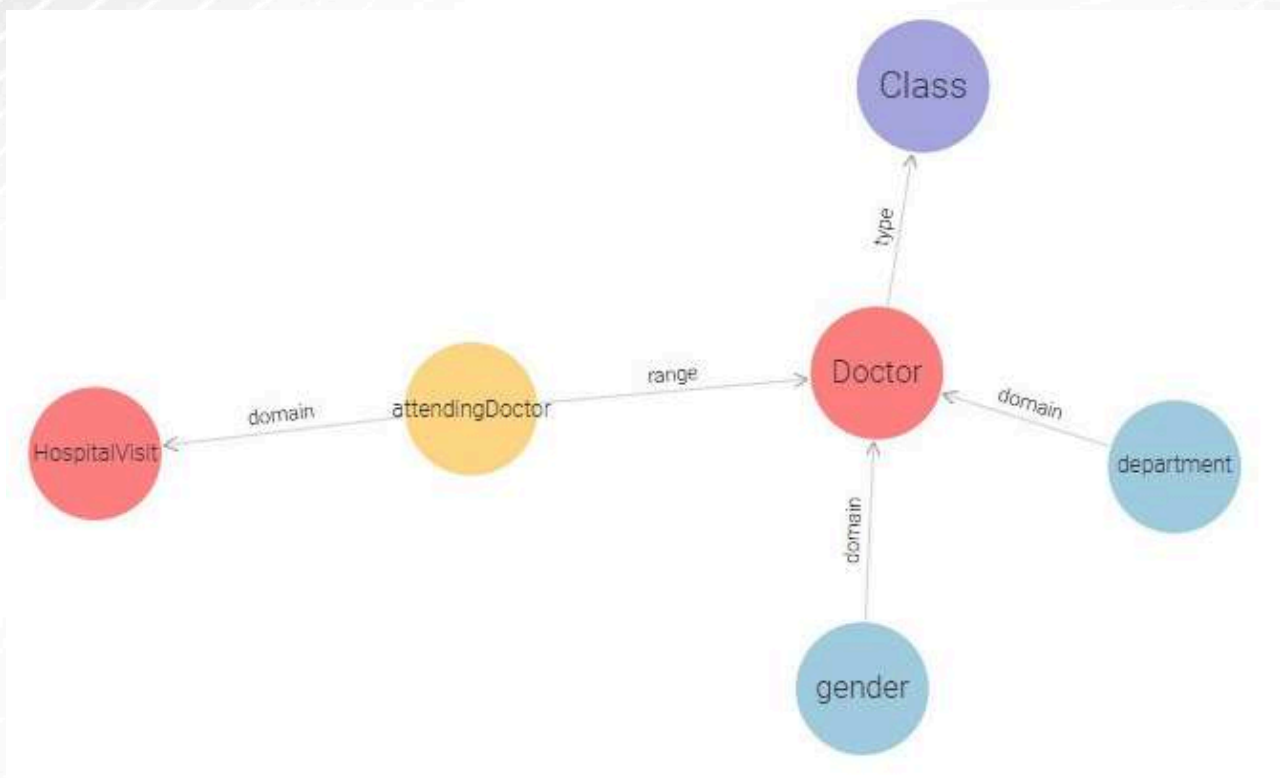
23

API Gateway

- Security & Policy: Enforces read-only access via a dedicated Policy Store.
- Request Routing: A central Router manages traffic between specialized agents and executors.
- Query Processing:
 - NL to SPARQL Agent: Translates Natural Language input into structured semantic queries.
 - SPARQL Executor: Finalizes and executes queries against the data store.
- Traffic Management: A Limit Enforcer prevents system congestion and maintains service availability.
- Monitoring: The Telemetry Sender pushes real-time logs and metrics to a Telemetry Collector for system auditing.

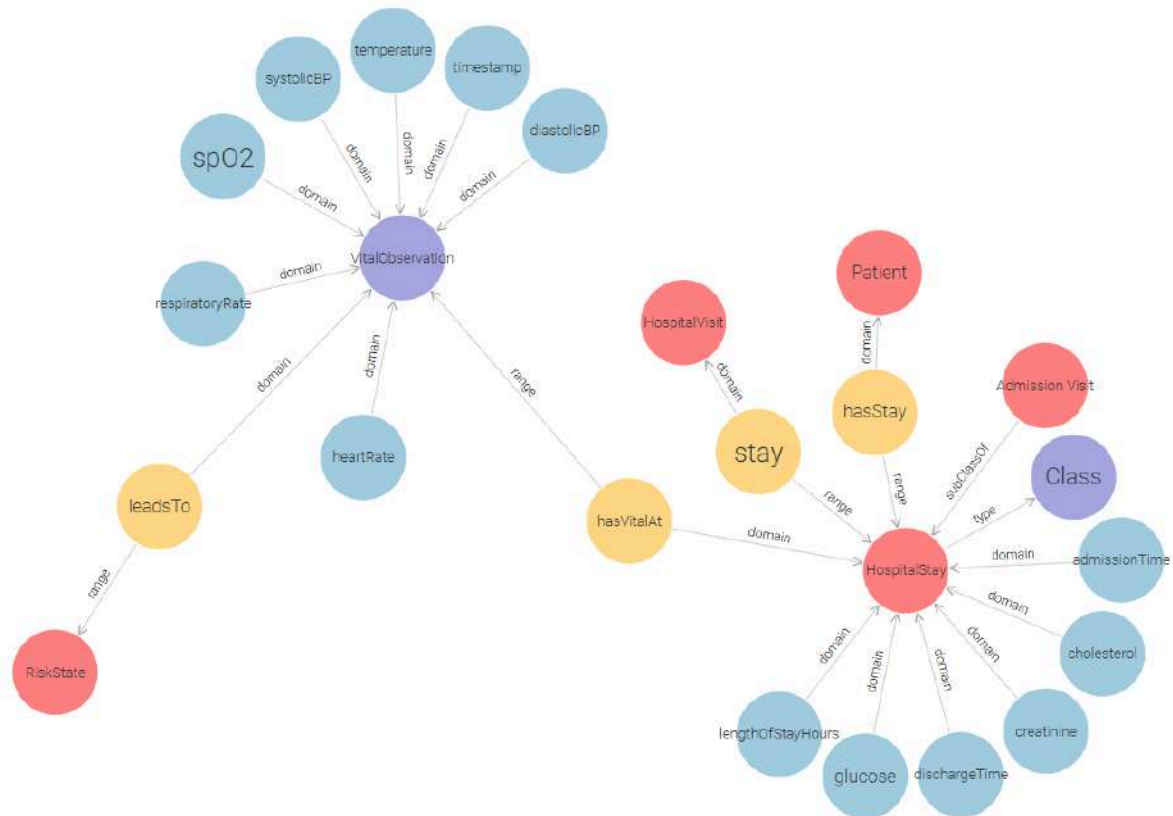
24

KG SCHEMA DESIGN

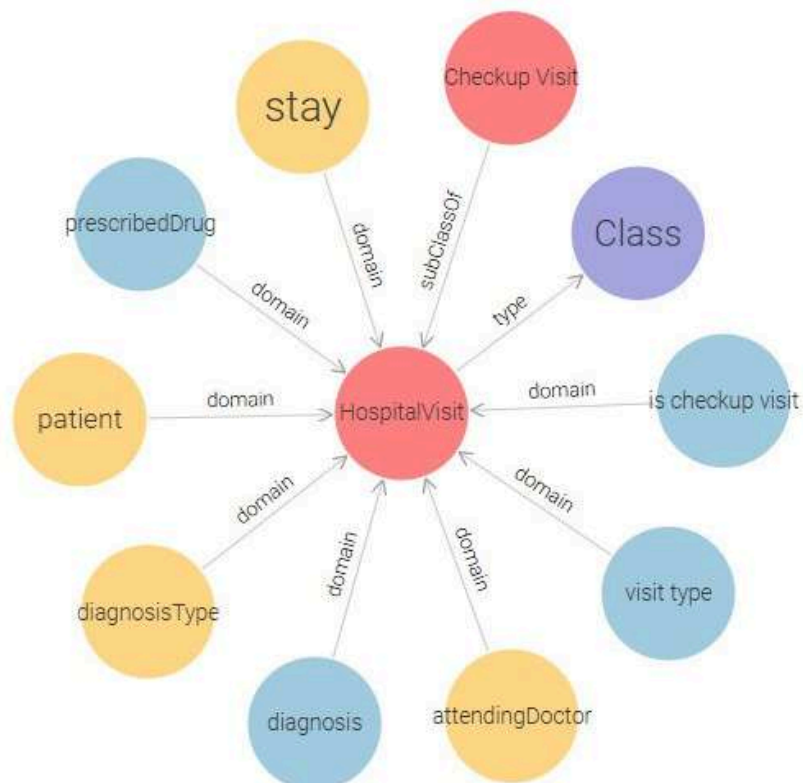


25

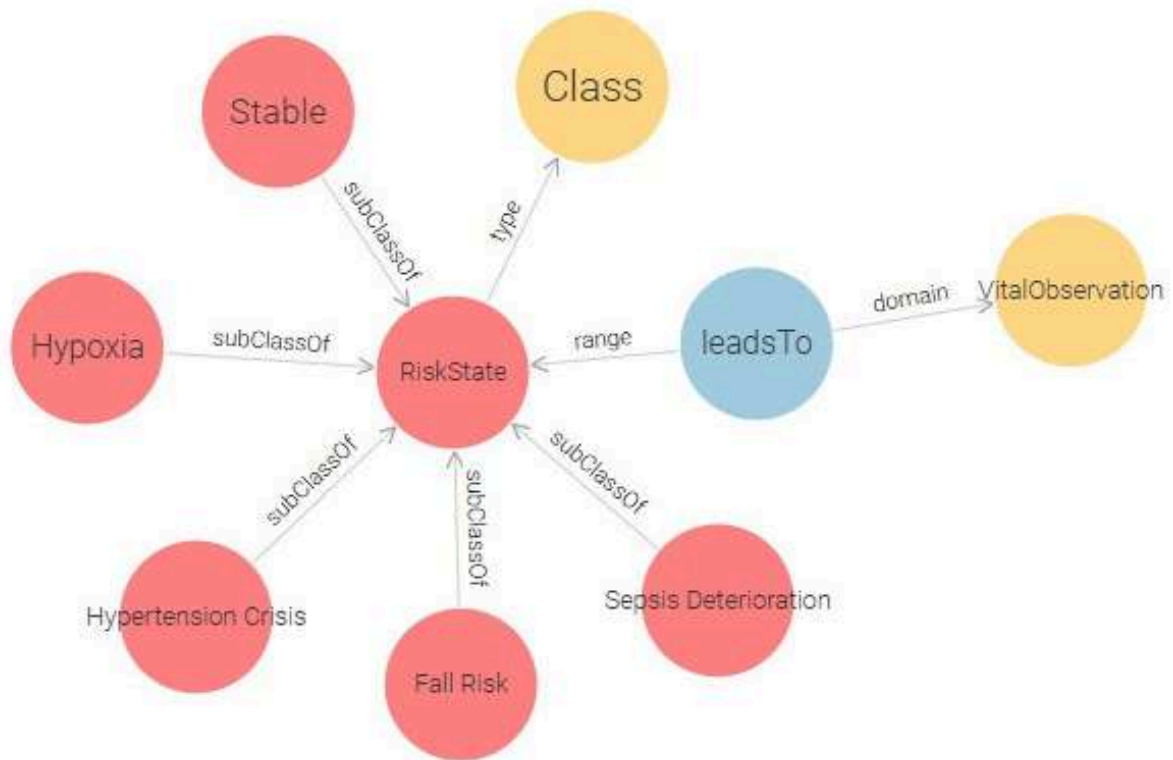
KG SCHEMA DESIGN



KG SCHEMA DESIGN

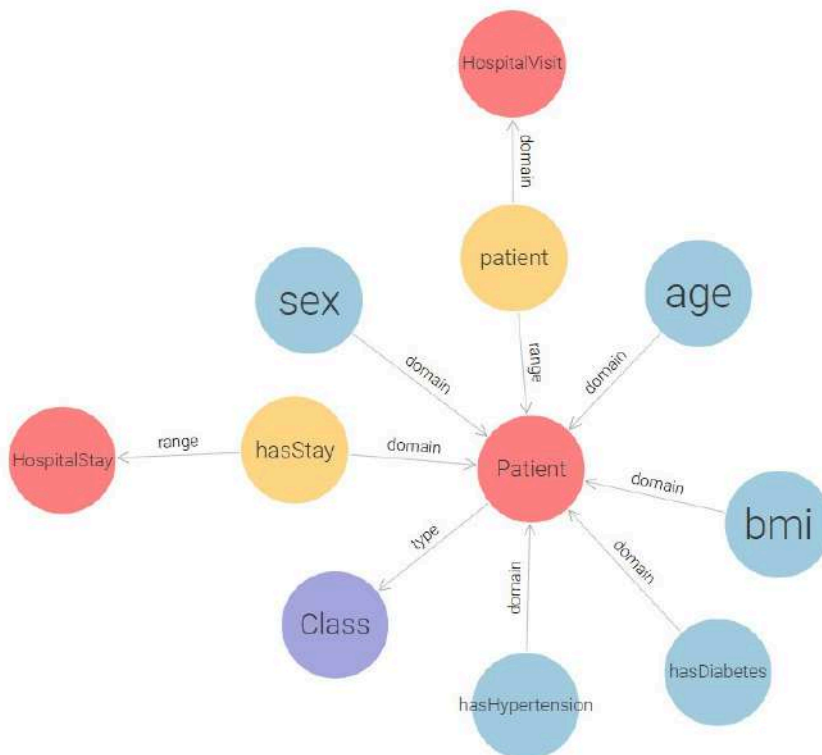


KG SCHEMA DESIGN



28

KG SCHEMA DESIGN



29

RESULT AND OUTPUT

```
(venv) PS C:\Main project\NL-2-SPARQL> python -u "c:\Main project\NL-2-SPARQL\agent.py"
SPARQL endpoint: http://localhost:3030/healthkg/sparql
Agent ready. (Wait 10s between complex questions to avoid Rate Limits)

Ask a healthcare question (or 'quit'): list 10 patients who have both diabetes and hypertension history
===== Human Message =====

list 10 patients who have both diabetes and hypertension history
===== Ai Message =====

Tool Calls:
  sparql_query_checker (70facd3f-0fba-438d-a787-560741bd8463)
Call ID: 70facd3f-0fba-438d-a787-560741bd8463
Args:
  query:
SELECT ?patient
WHERE {
  ?patient rdf:type ex:Patient .
  ?patient ex:hasDiabetes true .
  ?patient ex:hasHypertension true .
}
LIMIT 10
```

30

RESULT AND OUTPUT (CONTD...)

```
===== Tool Message =====
Name: sparql_query_checker

SELECT ?patient
WHERE {
  ?patient rdf:type ex:Patient .
  ?patient ex:hasDiabetes true .
  ?patient ex:hasHypertension true .
}
LIMIT 10

===== Ai Message =====
Tool Calls:
  sparql_query (4066fed7-c5d2-49f0-8f10-35a5069a6639)
Call ID: 4066fed7-c5d2-49f0-8f10-35a5069a6639
Args:
  query:
SELECT ?patient
WHERE {
  ?patient rdf:type ex:Patient .
  ?patient ex:hasDiabetes true .
  ?patient ex:hasHypertension true .
}
LIMIT 10
```

32

RESULT AND OUTPUT (CONTD...)

```
===== Tool Message =====
Name: sparql_query

patient values: http://example.org/health#P1062, http://example.org/health#P1551, http://example.org/health#P1575, http://example.org/health#P1624, http://example.org/health#P1795, http://example.org/health#P2113, http://example.org/health#P2463, http://example.org/health#P2640, http://example.org/health#P2991, http://example.org/health#P3011
===== Ai Message =====

[{'type': 'text', 'text': 'Here are 10 patients who have both diabetes and hypertension history:\nP1062, P1551, P1575, P1624, P1795, P2113, P2463, P2640, P2991, P3011', 'extras': {'signature': 'CuUBAb4+9vulGUOfzF0UyWMq7kUDHcIJMwWcdfKUHI/aLmMzk5/yx2BiKA0JWuJALhHiYE2KAoY5xe8Y2tYREQTggu8nMicAiLQkyDIlc8/hhQYeKq6BEp5Ny1wcQqY5asDATW9iIBNOTxeVTxsTk4Tvd66JSf931Tv40H2I2rYsBfzUDJ0fyKhSAXG11YycjojYn1TT6gU09tHnL/72gVDKfojo1UzhYeYjZEs0x2aI7Pm9N5DfH1RgoICB0qvSyhyUHwBXTy3jHmqcRowfErtPjQ5EBaD+wL6/cuSd+Xtb4W011c0cDA=='}}]

Ask a healthcare question (or 'quit'): 
```

33

```
Ask a healthcare question (or 'quit'): list patients above age 30
===== Human Message =====

list patients above age 30
===== Ai Message =====

Tool Calls:
  sparql_list_classes (6421bf0f-9f39-4d7e-86da-982534b953f1)
Call ID: 6421bf0f-9f39-4d7e-86da-982534b953f1
Args:
===== Tool Message =====
Name: sparql_list_classes

http://example.org/health#Disease, http://www.w3.org/2000/01/rdf-schema#Class, http://example.org/health#Symptom, http://example.org/health#Medication, http://example.org/health#Patient, http://example.org/health#Visit, http://example.org/health#Clinic, http://www.w3.org/1999/02/22-rdf-syntax-ns#Property, http://www.w3.org/ns/shacl#NodeShape
===== Ai Message =====

Tool Calls:
  sparql_schema (570d9879-565e-4f45-9e7b-05d869bf7f9b)
Call ID: 570d9879-565e-4f45-9e7b-05d869bf7f9b
Args:
  class_list: ex:Patient
```

34

```

===== Tool Message =====
Name: sparql_schema

Schema predicates (class -- predicate):
http://example.org/health#Patient -- http://www.w3.org/1999/02/22-rdf-syntax-ns#type
http://example.org/health#Patient -- http://www.w3.org/2000/01/rdf-schema#label
http://example.org/health#Patient -- http://example.org/health#hasDiagnosis
http://example.org/health#Patient -- http://example.org/health#hasAge
http://example.org/health#Patient -- http://example.org/health#hasGender
===== Ai Message =====

```

```

Tool Calls:
  sparql_query_checker (9489491b-15e5-48a9-a4fd-e5e59227d2a1)
Call ID: 9489491b-15e5-48a9-a4fd-e5e59227d2a1

```

```

  Args:
    query:
SELECT ?patient ?age WHERE {
  ?patient a ex:Patient ;
    ex:hasAge ?age .
  FILTER (?age > 30)
} LIMIT 50

```

```

===== Tool Message =====
Name: sparql_query_checker

```

```

SELECT ?patient ?age WHERE {
  ?patient a ex:Patient ;
    ex:hasAge ?age .
  FILTER (?age > 30)
} LIMIT 50

```

35

```

===== Ai Message =====

```

```

Tool Calls:
  sparql_query (1c8e4c53-1b60-4728-9509-68c44ac46e13)
Call ID: 1c8e4c53-1b60-4728-9509-68c44ac46e13

```

```

  Args:
    query:
SELECT ?patient ?age WHERE {
  ?patient a ex:Patient ;
    ex:hasAge ?age .
  FILTER (?age > 30)
} LIMIT 50

```

```

===== Tool Message =====

```

```

Name: sparql_query

```

```

Results from the knowledge graph:
patient=http://example.org/health#P001; age=34
patient=http://example.org/health#P002; age=58
patient=http://example.org/health#P004; age=42
patient=http://example.org/health#P005; age=31
patient=http://example.org/health#P006; age=63
patient=http://example.org/health#P007; age=45
patient=http://example.org/health#P009; age=37
patient=http://example.org/health#P010; age=50
patient=http://example.org/health#P012; age=39

```

36

```
74
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL GITLENS PORTS

-----
LABEL DISTRIBUTION:
- Stable: 2902
- Fall Risk (Hypotension): 418
- Hypertension Crisis: 888
- Deterioration (Sepsis): 1196
- Hypoxia: 596
-----

Labeled data saved to: ./dataset/vitals_labeled.ttl
(venv) PS C:\Main project\NL-2-SPARQL>
```

37

```
Generating Medical Metrics Report...
25/25 0s 2ms/step

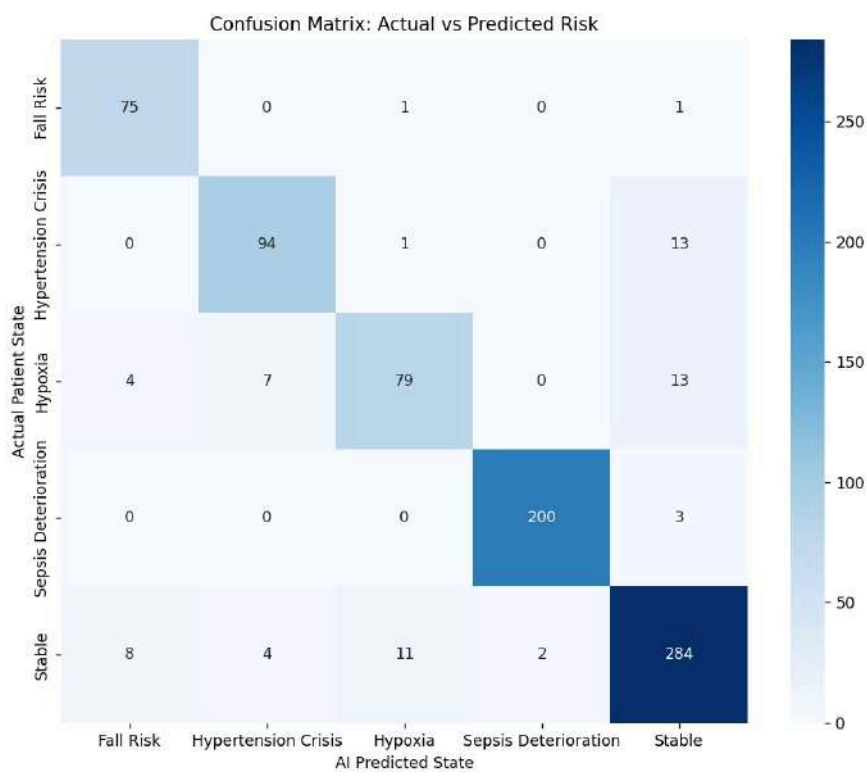
--- Classification Report ---
      precision    recall  f1-score   support

    Fall Risk         0.86      0.97      0.91         77
Hypertension Crisis    0.90      0.87      0.88        108
      Hypoxia         0.86      0.77      0.81        103
Sepsis Deterioration   0.99      0.99      0.99        203
      Stable         0.90      0.92      0.91        309

 accuracy              0.92         800
  macro avg              0.90         800
 weighted avg              0.91         800
```

38

EVALUATION METRICS



39

AI Assistant Context

Monitoring Patient: P1047

Active Stay ID: Stay_c28ced

Real-Time Vitals Entry

Heart Rate

75

-

+

Systolic BP

115

-

+

Diastolic BP

75

-

+

Resp Rate

16

-

+

Temp (°C)

37.00

-

+

SpO2 (%)

98

-

+

LOG VITALS & RUN AI

Vitals logged at simulated time: 2026-03-04T10:39:28.244876

AI requires 3 data points to predict. Collecting data...

Enter Patient ID:

P1047

Re-Admit / Start New Stay

AI Assistant Context

Monitoring Patient: P1047

Active Stay ID: Stay_c28ced

Real-Time Vitals Entry

Heart Rate

78

-

+

Systolic BP

98

-

+

Diastolic BP

65

-

+

Resp Rate

16

-

+

Temp (°C)

37.00

-

+

SpO2 (%)

98

-

+

LOG VITALS & RUN AI

Vitals logged at simulated time: 2026-03-04T14:39:28.244876

AI requires 3 data points to predict. Collecting data...

40

70

 AI Assistant Context

Monitoring Patient: P1047

Active Stay ID: Stay_c28ced

 Real-Time Vitals Entry

Heart Rate	Systolic BP	Diastolic BP
82 - +	85 - +	55 - +
Resp Rate	Temp (°C)	SpO2 (%)
18 - +	36.80 - +	97 - +

LOG VITALS & RUN AI

☒ Vitals logged at simulated time: 2026-03-04T18:39:28.244876

Patient is Stable (Confidence: 94.6%).

41

 AI Assistant Context

Monitoring Patient: P1047

Active Stay ID: Stay_c28ced

 Real-Time Vitals Entry

Heart Rate	Systolic BP	Diastolic BP
95 - +	75 - +	50 - +
Resp Rate	Temp (°C)	SpO2 (%)
20 - +	36.50 - +	95 - +

LOG VITALS & RUN AI

☒ Vitals logged at simulated time: 2026-03-04T22:39:28.244876

CRITICAL ALERT: Fall Risk DETECTED! (Confidence: 95.2%)

42

Heart Rate	Systolic BP	Diastolic BP
95 - +	132 - +	86 - +
Resp Rate	Temp (°C)	SpO2 (%)
30 - +	39.00 - +	85 - +

LOG VITALS & RUN AI

✓ Vitals logged at simulated time: 2026-03-03T20:47:12.087082

🚨 CRITICAL ALERT: Hypoxia DETECTED! (Confidence: 95.1%)

43



Health Care Monitor



Clinical Assistant Chat

 show medical history of P3199

 ✓ Response received!

Patient P3199's medical history includes:

- **Diagnosis:** Edema, **Prescribed Drug:** Furosemide
- **Diagnosis:** Shortness of Breath, **Prescribed Drug:** Furosemide
- **Diagnosis:** Heart Failure, **Prescribed Drug:** Furosemide
- **Diagnosis:** Heart Failure, **Prescribed Drug:** Furosemide

Ask about patient history or status... 

44

Health Care Monitor

Clinical Assistant Chat

show recent vital observations of P1047?

✓ Response received!

Here are the recent vital observations for Patient P1047:

Timestamp: 2026-03-04T22:39:28.244876

- Heart Rate: 95
- Systolic BP: 75
- Diastolic BP: 50
- SpO2: 95
- Respiratory Rate: 20
- Temperature: 36.5

Ask about patient history or status...

Health Care Monitor

Clinical Assistant Chat

- Temperature: 36.5
- Leads To: Fall Risk
- Risk Confidence: 0.96213746

Timestamp: 2026-03-04T18:39:28.244876

- Heart Rate: 82
- Systolic BP: 85
- Diastolic BP: 55
- SpO2: 97
- Respiratory Rate: 18
- Temperature: 36.8
- Leads To: Stable
- Risk Confidence: 0.946041

Timestamp: 2026-03-04T14:39:28.244876

Ask about patient history or status...

45

ASSUMPTIONS

- It is assumed that historical data snapshots are available up to the query timestamp to perform link prediction for future events.
- Historical medical events will exhibit recurring patterns over time, and using these "vital patterns" will give significant insight into future health outcomes.
- The proposed method assumes that the "answer space" for any natural language query is entirely contained within the nodes and relations of the underlying knowledge graph. i.e for any query outside the span of the contents of the Knowledge graph will render no result.

46

HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements

- Processor : Quad-core CPU (Intel i5 11th Gen or AMD Ryzen 7 5800X)
- RAM : 16GB
- Operating System : Ubuntu 22.04 LTS or Windows 10

Software Requirements

- Large Language Models - Gemini model (like Gemini-2.5-flash)
 - RDF Triple Store or Graph Database - Apache Jena Fuseki
 - LangChain Library
 - Python 3.9+
 - Tensorflow and Keras
 - VS Code
 - Dataset - <https://www.kaggle.com/datasets/uom190346a/synthetic-clinical-tabular-dataset>
- 47• (Obtained from Kaggle)

WORK BREAKDOWN AND RESPONSIBILITIES

Reno Jacob Zachariah

- NL to SPARQL agent construction
- Labeling of Vital observations with risk_states
- Feature Extraction and preprocessing for training
- LSTM training
- Training evaluation

Reuben Mathew Shaji

- Synthetic Data Preparation - Hospital Files , Doctor file
- ETL (Extract Transform Load) Pipeline construction
- Chatbot Integration With UI
- Logger development
- Backend and API development

WORK BREAKDOWN AND RESPONSIBILITIES

Rohan Cijoy

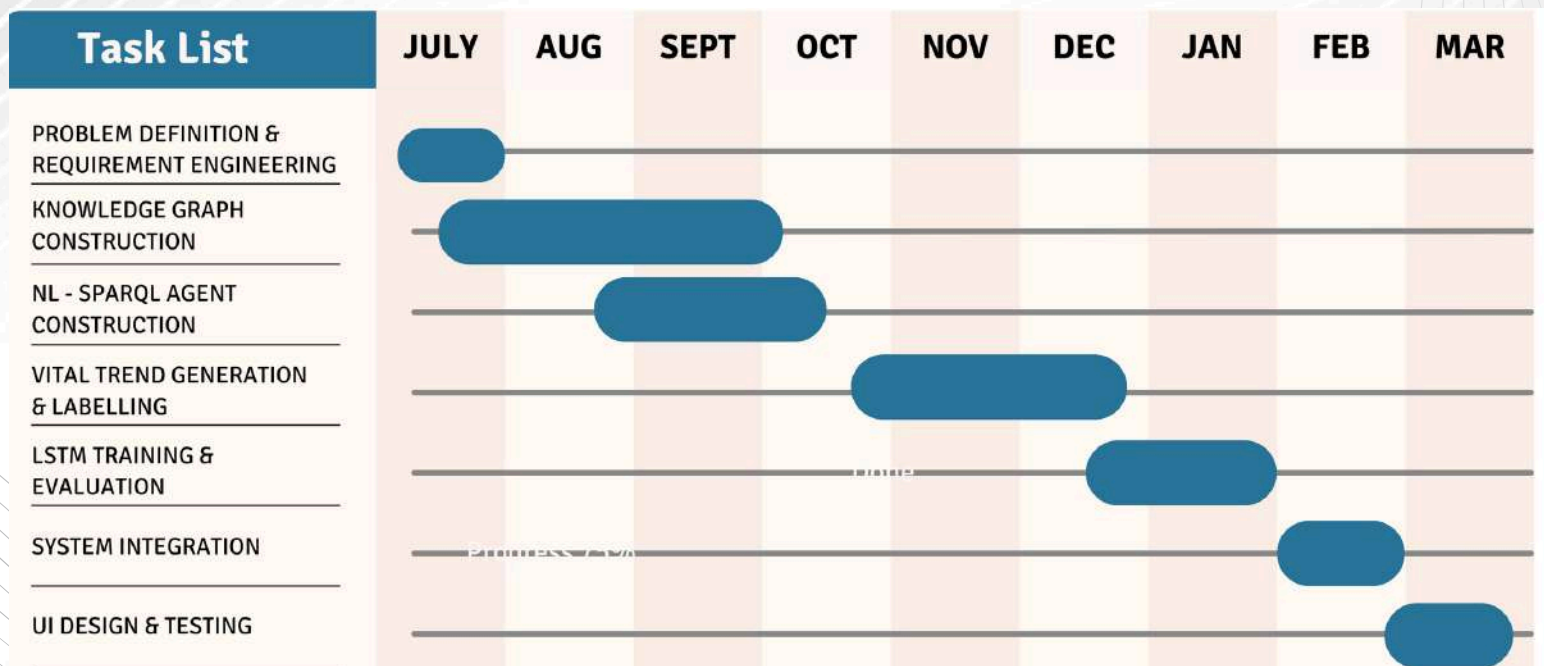
- Synthetic Data Preparation - Patient data , stays file
- UI development
- Testing NL to SPARQL agent
- Refining the Agent Prompt for better performance based on Testing
- SHACL (shape constraint language) development.

Vyshnav A K

- Synthetic Data Preparation - Vital Observations
- Schema construction for KG
- Vital trends generation for temporal pattern
- Testing the Synthetic Data on Jena Fuseki for identifying node connection issues
- LSTM integration for real time alerts

49

GANTT CHART



50

RISKS AND CHALLENGES

- Data sparsity within specific time intervals can cause the model to hallucinate patterns that do not exist.
- Efficiently storing and querying large-scale graph snapshots across thousands of timestamps is computationally expensive.
- A major risk is Schema Mapping Error, where the LLM correctly understands the user's intent but maps it to the wrong RDF predicate (e.g., using `ex:hasHeartRate` instead of `ex:heartRateValue`).
- LSTMs are highly sensitive to sampling frequency. If a model trained on 4-hour intervals receives data every 30 minutes during a crisis, its sliding window will cover only 1.5 hours instead of the expected 12. This Temporal Out-of-Distribution (OOD) error causes the model's "internal clock" to mismatch with real-world entry speeds, potentially failing to recognize slow-moving physiological deterioration.

51

CONCLUSION

- Allows healthcare professionals to query and analyze structured EHR data using everyday language.
- Uses a healthcare-specific Knowledge Graph and an LLM-powered agent to translate questions into precise, secure database queries.
- Delivers results in clear, user-friendly chat format.
- Enables medical staff to make informed, data-driven decisions quickly without needing technical expertise.
- Features a read-only design, performance safeguards, and a user-friendly interface to ensure trustworthy and accessible analytics.
- Allows clinicians to get predictive analytics over vital patterns with confidence score which is triggered on vital addition

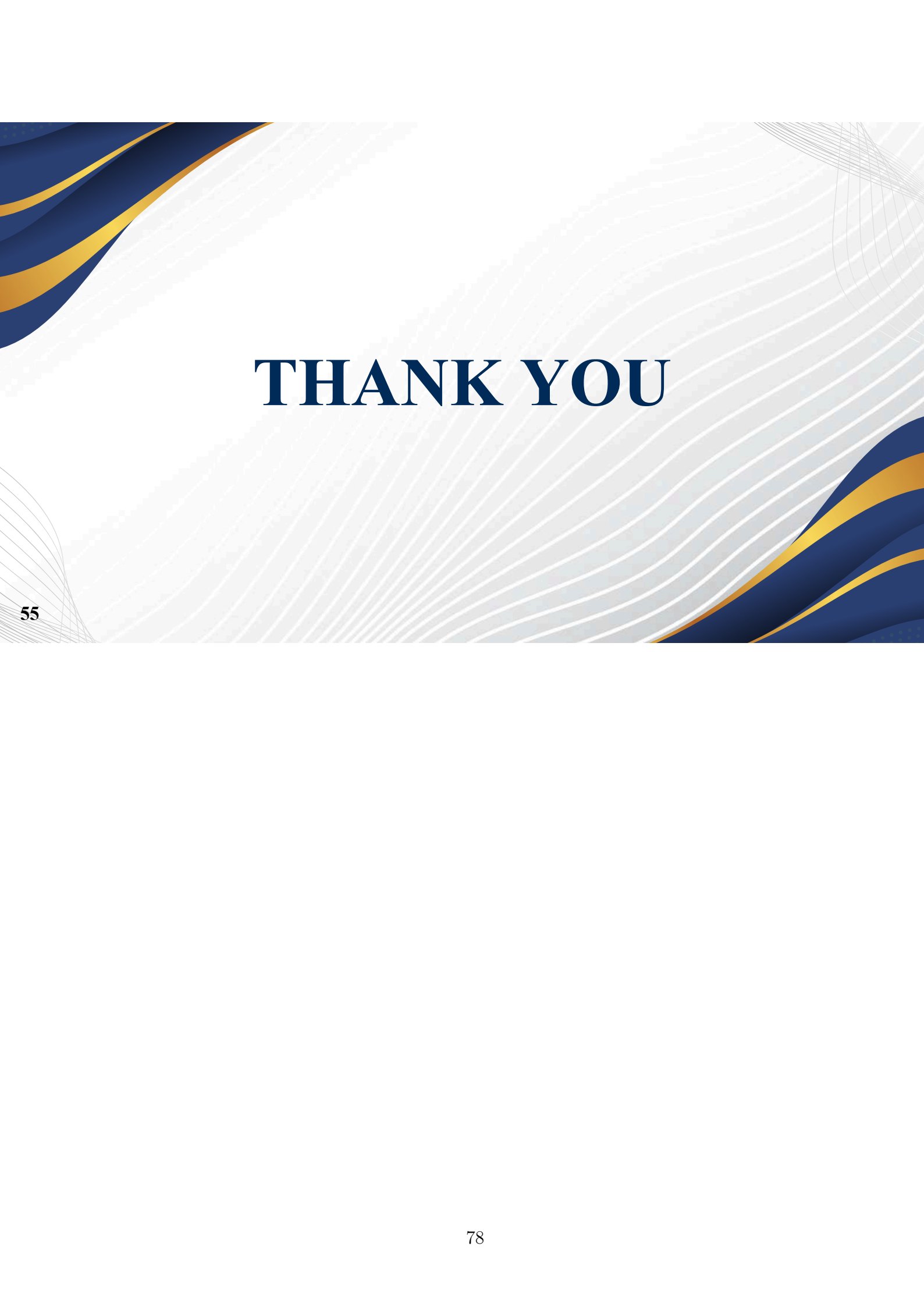
52

REFERENCES

- [1] S. Liang, K. Stockinger, T. M. de Farias, M. Anisimova, and M. Gil, “Querying knowledge graphs in natural language,” *Journal of Big Data*, vol. 8, no. 3, pp. 1–23, 2021.
- [2] A. Xie, “Deep representation learning for risk prediction in electronic health records using self-supervised methods,” *Preprints.org*, 2026, not peer-reviewed version.
- [3] J. Park, Y. Cho, H. Lee, J. Choo, and E. Choi, “Knowledge graph-based question answering with electronic health records,” *Proceedings of Machine Learning Research*, vol. 149, pp. 1–17, 2021.
- [4] J. D’Abramo, A. Zugarini, and P. Torroni, “Investigating large language models for text-to-SPARQL generation,” in *Proceedings of the Association for Computational Linguistics*, 2024.
- [5] Y. Pan, M. Wang, L. Lu, R. Lamsal, E. Pärn, S. Zlatanova, and I. Brilakis, “LLM-enabled multi-agent framework for natural language interaction with graph-based digital twins,” *Automation in Construction*, vol. 183, p.106791, 2026.

REFERENCES

- [6] R. Chaturvedi and B. Di Eugenio, “Temporal knowledge graph extraction and modeling across multiple documents for health risk prediction,” in *Companion Proceedings of the ACM Web Conference 2024*. ACM, 2024, pp. 1181–1185.
- [7] M. Postiglione, D. Bean, Z. Kraljevic, R. J. Dobson, and V. Moscato, “Predicting future disorders via temporal knowledge graphs and medical ontologies,” *IEEE Journal of Biomedical and Health Informatics*, vol. 28, no. 7, pp. 4238–4249, 2024.
- [8] X. Zhao, S. Liu, S.-Y. Yang, and C. Miao, “MedRAG: Enhancing retrieval-augmented generation with knowledge graph-elicited reasoning for healthcare copilot,” in *Proceedings of the ACM Web Conference 2025*. ACM, 2025.
- [9] M. Yang, Z. Peng, C. van Pul, P. Andriessen, K. Dong, D. Silvertand, J. Li, C. Liu, and X. Long, “Continuous prediction and clinical alarm management of late-onset sepsis in preterm infants using vital signs from a patient monitor,” *Computer Methods and Programs in Biomedicine*, vol. 255, p. 108335, 2024.
- [10] L. J. Liu, V. Ortiz-Soriano, J. A. Neyra, and J. Chen, “KIT-LSTM: Knowledge-guided time-aware LSTM for continuous clinical risk prediction,” in *2022 IEEE International Conference on Bioinformatics and Biomedicine (BIBM)*. IEEE, 2022, pp. 1–8.



THANK YOU

Appendix B: Vision, Mission, Programme Outcomes and Course Outcomes

Vision, Mission, Programme Outcomes and Course Outcomes

Institute Vision

To evolve into a premier technological institution, moulding eminent professionals with creative minds, innovative ideas and sound practical skill, and to shape a future where technology works for the enrichment of mankind.

Institute Mission

To impart state-of-the-art knowledge to individuals in various technological disciplines and to inculcate in them a high degree of social consciousness and human values, thereby enabling them to face the challenges of life with courage and conviction.

Department Vision

To become a centre of excellence in Computer Science and Engineering, moulding professionals catering to the research and professional needs of national and international organizations.

Department Mission

To inspire and nurture students, with up-to-date knowledge in Computer Science and Engineering, ethics, team spirit, leadership abilities, innovation and creativity to come out with solutions meeting societal needs.

Programme Outcomes (PO)

Engineering Graduates will be able to:

PO1. Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop solutions to complex engineering problems.

PO2. Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development.

PO3. Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for public health and safety, whole-life cost, net zero carbon, culture, society and environment as required.

PO4. Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis and interpretation of data to provide valid conclusions.

PO5. Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering and IT tools, including prediction and modelling, recognizing their limitations to solve complex engineering problems.

PO6. The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for their impact on sustainability with reference to economy, health, safety, legal framework, culture and environment.

PO7. Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national and international laws.

PO8. Individual and Collaborative Team Work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO9. Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, and make effective presentations considering cultural, language and learning differences.

PO10. Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making, and apply these to one's own work, as a member and leader in a team, and to manage projects in multidisciplinary environments.

PO11. Life-long Learning: Recognize the need for, and have the preparation and ability for (i) independent and life-long learning, (ii) adaptability to new and emerging technologies, and (iii) critical thinking in the broadest context of technological change.

Programme Specific Outcomes (PSO)

A graduate of the Computer Science and Engineering Program will demonstrate:

PSO1: Computer Science Specific Skills

The ability to identify, analyze and design solutions for complex engineering problems in multidisciplinary areas by understanding the core principles and concepts of computer science and thereby engage in national grand challenges.

PSO2: Programming and Software Development Skills

The ability to acquire programming efficiency by designing algorithms and applying standard practices in software project development to deliver quality software products meeting the demands of the industry.

PSO3: Professional Skills

The ability to apply the fundamentals of computer science in competitive research and to develop innovative products to meet the societal needs thereby evolving as an eminent researcher and entrepreneur.

Course Outcomes (CO)

Course Outcome 1: Model and solve real world problems by applying knowledge across domains (Cognitive knowledge level: Apply).

Course Outcome 2: Develop products, processes or technologies for sustainable and

socially relevant applications (Cognitive knowledge level: Apply).

Course Outcome 3: Function effectively as an individual and as a leader in diverse teams and to comprehend and execute designated tasks (Cognitive knowledge level: Apply).

Course Outcome 4: Plan and execute tasks utilizing available resources within timelines, following ethical and professional norms (Cognitive knowledge level: Apply).

Course Outcome 5: Identify technology/research gaps and propose innovative/creative solutions (Cognitive knowledge level: Analyze).

Course Outcome 6: Organize and communicate technical and scientific findings effectively in written and oral forms (Cognitive knowledge level: Apply).

Appendix C: CO-PO-PSO Mapping

COURSE OUTCOMES

SNO	DESCRIPTION	BLOOM'S TAXONOMY LEVEL
1	CO1: Model and solve real world problems by applying knowledge across domains.	Apply
2	Develop products, processes or technologies for sustainable and socially relevant application.	Apply
3	Function effectively as an individual and as a leader in diverse teams and to comprehend and execute designated tasks.	Apply
4	Plan and execute tasks utilizing available resources within timelines, following ethical and professional norms.	Apply
5	Identify technology/research gaps and propose innovative/creative solutions.	Analyze
6	Organize and communicate technical and scientific findings effectively in written and oral forms.	Apply

MAPPING COURSE OUTCOMES (COs) – PROGRAM OUTCOMES (POs) AND PROGRAM SPECIFIC OUTCOMES (PSOs)

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2	PSO3
CO1	2	2	2	1		2	2	1	1	1	1	2	2	2	2
CO2	2	2	2		1	3	3	1	1	1	1	2	2		
CO3							3	2	2	1					3
CO4			2		3	2	2	3	2		3	3			
CO5	2	3	3	1	2					1	3				
CO6			2			2	2	3	1	1		3			

JUSTIFICATIONS FOR CO-PO MAPPING

MAPPING	LOW / MEDIUM / HIGH	JUSTIFICATION
101003/ CS822U.CO1- PO1	Medium	Applies engineering fundamentals to model and solve real-world problems.
101003/ CS822U.CO1- PO2	Medium	Involves problem analysis and identification of solutions.
101003/ CS822U.CO1- PO3	Medium	Requires system/design development to meet requirements.
101003/ CS822U.CO1- PO4	Low	Limited experimental validation and investigation.
101003/ CS822U.CO1- PO5	Medium	Uses modern tools, simulation software, and platforms.
101003/ CS822U.CO1- PO6	Medium	Solutions may impact societal/industrial context.
101003/ CS822U.CO1- PO7	Medium	Sustainable considerations during solution development.
101003/ CS822U.CO1- PO8	Low	Ethical considerations during implementation.
101003/ CS822U.CO1- PO9	Low	Team collaboration during modeling phase.
101003/ CS822U.CO1- PO10	Low	Communication of modeling results.
101003/ CS822U.CO1- PO11	Low	Resource and time planning during execution.
101003/ CS822U.CO1- PO12	Medium	Independent learning and domain integration.
101003/ CS822U.CO1- PSO1	Medium	Ability to identify, analyze and design solutions to identify technically and economically feasible problems.
101003/ CS822U.CO1- PSO2	Medium	Designing algorithms and applying standard practices in software project development.

MAPPING	LOW / MEDIUM / HIGH	JUSTIFICATION
101003/ CS822U.CO1- PSO3	Medium	Fundamentals of computer science in competitive research can be applied to identify technically feasible problems.
101003/ CS822U.CO2- PO1	Medium	Identify and survey the relevant literature by applying knowledge of mathematics, science, and engineering fundamentals.
101003/ CS822U.CO2- PO2	Medium	Identify, formulate, review research literature, and analyze complex engineering problems.
101003/ CS822U.CO2- PO3	Medium	Design solutions for complex engineering problems based on the relevant literature.
101003/ CS822U.CO2- PO5	Low	Identify and survey relevant literature and become familiar with software development tools.
101003/ CS822U.CO2- PO6	High	Create, select, and apply appropriate techniques and resources using surveyed literature.
101003/ CS822U.CO2- PO8	High	Apply ethical principles and commit to professional ethics based on literature review.
101003/ CS822U.CO2- PO9	Low	Identify and survey relevant literature as a team.
101003/ CS822U.CO2- PO10	Low	Communicate findings from literature review to the engineering fraternity.
101003/ CS822U.CO2- PO11	Low	Demonstrate engineering and management principles through literature survey.
101003/ CS822U.CO2- PO12	Medium	Encourage independent and lifelong learning through literature review.
101003/ CS822U.CO2- PSO2	Medium	Application of domain-specific tools for real-world development.
101003/ CS822U.CO3- PO9	High	Function effectively as an individual and as a member or leader in teams.

MAPPING	LOW / MEDIUM / HIGH	JUSTIFICATION
101003/ CS822U.CO3- PO10	Medium	Communicate effectively with the engineering community and society.
101003/ CS822U.CO3- PO11	Medium	Demonstrate engineering requirement analysis and design methodologies.
101003/ CS822U.CO3- PO12	Low	Recognize the need for independent and life-long learning in technological change.
101003/ CS822U.CO3- PSO3	High	Apply computer science fundamentals in competitive research and design methodologies.
101003/ CS822U.CO4- PO3	Medium	Planning impacts design and execution.
101003/ CS822U.CO4- PO8	High	Strong alignment with professional ethics.
101003/ CS822U.CO4- PO9	Medium	Team coordination in execution.
101003/ CS822U.CO4- PO10	Medium	Reporting and documentation.
101003/ CS822U.CO4- PO11	High	Emphasis on project management and scheduling.
101003/ CS822U.CO4- PO12	High	Continuous improvement and structured execution.
101003/ CS822U.CO4- PSO3	Medium	Execution of computing projects in a structured manner.
101003/ CS822U.CO5- PO1	Medium	Requires strong foundational knowledge.
101003/ CS822U.CO5- PO2	High	High-level analytical thinking and gap identification.
101003/ CS822U.CO5- PO3	High	Innovative system design solutions.
101003/ CS822U.CO5- PO4	Low	Some investigative research work involved.

MAPPING	LOW / MEDIUM / HIGH	JUSTIFICATION
101003/ CS822U.CO5- PO5	Medium	Use of advanced tools and technologies.
101003/ CS822U.CO5- PO10	Low	Proposal presentation and justification.
101003/ CS822U.CO5- PO12	High	Self-learning, research orientation and creativity.
101003/ CS822U.CO5- PSO1	High	Deep technical and research competence in CSE domain.
101003/ CS822U.CO6- PO3	Medium	Prepare design solutions for complex engineering problems and create technical reports.
101003/ CS822U.CO6- PO5	Medium	Apply appropriate techniques and modern engineering tools in preparing reports.
101003/ CS822U.CO6- PO8	Medium	Prepare reports by applying ethical principles and professional responsibilities.
101003/ CS822U.CO6- PO9	High	Deliver presentations effectively as individuals and team members.
101003/ CS822U.CO6- PO10	Low	Alignment with communication skills.
101003/ CS822U.CO6- PO11	Low	Apply engineering and management principles in report preparation.
101003/ CS822U.CO6- PSO3	High	Professional technical documentation and presentation in computing domain.